From Experience to Method: Evidence-Governed Continual Evolution of LLM Research Systems

Sze Chun Yiu
Stockholm University
sze-chun.yiu@fysik.su.se

11 August 2026

Abstract

Long-running large-language-model research agents produce more than candidate answers. They generate longitudinal evidence about how research is conducted: which representations are chosen, which prior experiences are retrieved or missed, which methods are repeated, where local results fail to connect to a root problem, and how verification or provenance defects alter the trajectory. Existing work has shown that agents can learn from episodic memory, induce reusable workflows and skills, and modify their own code, while recent behavioural studies show that workflow design can materially shape autonomous research. We ask a stricter question: *can the research method itself evolve from these trajectories under an evidence regime that prevents the system from certifying its own improvement?*

We introduce RAKL v3 as a persistent external research substrate in which immutable task episodes, versioned lessons, failures, operators, strategy motifs, problem-conditioned fibres, gluing obstructions and meta-method variants are overlapping views of one typed state. Experience may change future search priority, but cannot by itself mint theorem, tool, gluing or framework authority. We use a live multi-agent programme spanning six unsolved Millennium Prize Problems as a longitudinal case study. Early retrospective observations identify recurring process pathologies including missed cross-problem memory, surrogate quantities that do not preserve root-critical coordinates, locally valid lemmas with unresolved interface obligations, source-completeness failures, chronology repairs and evaluator/CI identity drift. These observations are hypothesis-generating case evidence, not proof that the framework has improved.

To make learning and recursive improvement measurable, we define a seven-axis retained-growth vector over knowledge, operators, experience patterns, obstructions, relations, successful paths and meta-methods; per-process telemetry for the RAKL method surface; experience-to-lesson/tool conversion and repeated-failure metrics; and a four-arm causal-attribution benchmark comparing the same base model under `MODEL_ONLY`, static `RAKL_RESET`, budget-matched `RAKL_SHAM_MEMORY` and persistent `RAKL_LEARNING` conditions. This separates static architecture lift, continual-experience lift and memory-content-specific lift rather than crediting every successful RAKL-assisted run to the framework.

We further specify a governed upgrade protocol for RAKL 3.x. Framework changes are classified as implementation, workflow or constitutional changes; challengers are content-identified and evaluated against frozen parent states, model/tool contracts and resource ceilings; fresh transfer tasks are hidden from the proposer; evaluators and thresholds are protected from candidate modification within an evaluation epoch; previous variants remain rollback targets; operator overrides are recorded without receiving evolution-evidence credit; and post-promotion attestation is separated from pre-promotion authorization. A repository-native coding-agent harness routes agents from a concise instruction map to a machine-readable method-state manifest, the upgrade protocol, metrology contracts and protected gates.

The present paper therefore contributes the deployed v3 experience substrate, longitudinal case-study methodology, integrated measurement-only metrology and causal-attribution instrumentation, and a preregistered evaluation and upgrade-governance programme. The

current v3 authority paths have been hardened with protected content-bound attestations and subject binding at the implementation level; this internal hardening is not represented as independent external assurance. We do not claim that RAKL 3.1 or later outperforms 3.0 on fresh research tasks; that claim is reserved for prospective matched and protected assurance evidence.

1 Research systems should learn how to research

The standard unit of evaluation for an AI agent is a task outcome. A task is given, the agent acts, and the system is scored on success, correctness, reward or cost. This abstraction is appropriate for many deployed agents, but it discards much of the evidence produced by open-ended research. A research run contains a sequence of representation choices, decompositions, literature searches, retrieved analogies, rejected tools, conjectures, falsifiers, source checks, local proofs, failed bridges and residual questions. Even when the root problem remains unsolved, these intermediate events reveal which research methods were useful, which failed, and why.

This distinction matters because long-horizon autonomous research is not well described by repeated independent benchmark queries. A recent behavioural case study of roughly one hundred sequential architecture experiments found a rapid-gain phase, an extended saturation wall and later recovery after the action surface was expanded; it also traced part of the observed greedy search behaviour to the workflow itself rather than to the underlying model alone [1]. The result suggests that the research harness is not merely an implementation detail. It can shape what hypotheses are generated, how risk is taken and when the system escapes a local research regime.

At the same time, a growing family of agent systems learns from externalized experience. Reflexion stores verbal reflections [2]; ExpeL extracts reusable insights from collections of trajectories [3]; Voyager accumulates an executable skill library [4]; Agent Workflow Memory induces reusable routines [5]; and more recent systems learn hierarchical skills or dual streams of experience and skill knowledge [6, 7]. These systems establish that persistent non-parametric experience can improve later behaviour. They also expose a new control problem. External memory can create negative transfer, retrieval competition and consolidation failure. In sequential settings, the continual-learning bottleneck may move from model weights to memory representation and access [8]; repeated LLM consolidation can even corrupt useful experience, motivating raw episodic preservation as a first-class design requirement [9].

A separate line of work asks agents to improve the agent architecture itself. Automated Design of Agentic Systems searches over agent programs [13]; Gödel Agent and related systems permit self-referential policy modification [15]; and Darwin Gödel Machine maintains an open-ended archive of self-modifying coding agents selected through empirical evaluation [16]. These approaches make recursive improvement operational. They also sharpen an unanswered engineering question for research systems: *what evidence licenses the claim that a self-modification is an improvement when the same system can propose the modification, influence the evaluator, select the tasks and rewrite the state from which future evidence is interpreted?*

Our previous RAKL work addressed a different layer of this problem. Paper 4 introduced a verified-discovery architecture for LLM-mediated mathematical research in which specification alignment, theorem truth, novelty, research value and verifier trust are separate authority coordinates; proposal generation cannot promote itself to theorem authority; and computation, proof and novelty are kept distinct [23]. The present work asks the next question. Once research trajectories are preserved under such an authority regime, can they be used to improve the *method of research itself* without collapsing observed performance, deployment status and epistemic authority into one scalar notion of "better"?

1.1 Central claim and evidence boundary

Our central claim is architectural and methodological:

> A long-running LLM research system can treat its own research trajectories as typed evidence for method improvement if raw episodes are preserved, diagnoses and abstractions remain defeasible, method changes are evaluated as content-identified challengers against frozen matched baselines and fresh transfer tasks, and promotion is separated from both proposal generation and operational deployment.

This paper supports that claim at three different evidence levels that must not be conflated.

1. **Implemented architecture.** RAKL v3 contains executable objects for immutable task episodes, versioned lessons, failure revisions, problem fibres, local-to-global gluing, experience-conditioned routing, vector saturation, problem-novelty classification, branching evolution variants and matched experience benchmarks.
2. **Retrospective/live case-study evidence.** A multi-agent mathematical research programme in the separate `RAKL_math` repository has already produced auditable examples of retrieval misses, representation failures, source-boundary failures, local-to-global interface gaps, chronology repairs and assurance defects. These observations motivate hypotheses about research-process weaknesses. Because much of the programme predates v3 telemetry, we treat these observations as qualitative and partly retrospective.
3. **Prospective claims not yet licensed.** We preregister how later RAKL variants such as 3.1 and 3.2 should be evaluated. Until those matched development and fresh-assurance trials are run, we do not claim a measured general improvement in research capability, efficiency or scientific discovery rate.

The distinction is deliberate. A paper about recursive self-improvement is especially vulnerable to circular evidence. If the architecture is allowed to define success after seeing the outcome, or if a framework version is called improved merely because it was merged, the empirical claim becomes unfalsifiable.

1.2 Research as four coupled loops

We model the persistent research system as a state R_t coupled to a proposal generator with parameters θ. For problem input P_t,

$$(S_t, \tau_t) = \mathrm{Driver}_\theta(P_t, R_t), \qquad R_{t+1} = \mathrm{Learn}(R_t, \tau_t),$$

where S_t is the task output and τ_t is the externally recorded public research trajectory. The base model parameters may remain fixed while behaviour changes through the evolving external state.

The system is organized around four coupled loops:

$$\text{information} \to \text{knowledge},$$

$$\text{problem} \to \text{solution},$$

$$\text{experience} \to \text{method},$$

$$\text{RAKL} \to \text{better candidate RAKL}.$$

The first two loops are conventional research operations. The third turns task experience into reusable but bounded method knowledge. The fourth is a meta-loop that proposes changes to the research architecture itself. Paper 5 focuses on the boundary between the third and fourth loops.

1.3 Why the unit of analysis cannot be "success" alone

Open research has sparse and delayed terminal rewards. A Millennium Prize Problem may remain unsolved for the duration of the study, while a run can still produce valuable information by refuting a method family, identifying an exact missing theorem, recovering a relevant prior result, or proving a scoped local lemma. Conversely, a locally correct lemma may create little root progress if the interface needed to connect it to the target remains exactly the hard part.

We therefore treat research progress as a vector of outcomes rather than a single score. Relevant coordinates include valid residual contraction, repeated-failure avoidance, retrieval coverage, correct rejection of unsafe transfer, proof or falsifier quality, detection of gluing/interface failure, resource use and authority preservation. A framework upgrade is admissible only if soft gains do not purchase violations of hard authority invariants.

This leads to the core methodological separation used throughout the paper:

$$\boxed{\text{DEPLOYMENT} \neq \text{PERFORMANCE EVIDENCE} \neq \text{METHOD AUTHORITY}}.$$

Code may be merged. A benchmark may improve. A method may even receive fresh assurance. These are different facts and are represented separately.

2 A longitudinal observatory for machine research

2.1 The live mathematical case study

The observational substrate is a live research programme in `SzeChunYiu/RAKL_math`. The repository is deliberately separated from the reusable RAKL framework repository. The application repository contains problem contracts, context fibres, research DAGs, source packets, candidate artifacts, falsifiers, reviews, traces and problem-specific tests for six unsolved Millennium Prize Problems: P versus NP, the Riemann Hypothesis, Navier–Stokes existence and smoothness, Yang–Mills existence and mass gap, the Hodge Conjecture, and the Birch and Swinnerton–Dyer Conjecture. A cross-problem lane studies transfer and recurring process failures across these domains.

At the initial Paper 5 freeze, the active scheduled programme contains nine hourly research lanes: P versus NP; an analytic Riemann-Hypothesis lane; Navier–Stokes regularity and blow-up lanes; constructive and spectral/RG Yang–Mills lanes; a Hodge deformation lane; an analytic BSD lane; and a cross-Millennium transfer/meta-observer lane. Complementary spectral RH, motive-oriented Hodge and Selmer/Iwasawa BSD lanes are preserved but paused. The schedule is an operational fact, not a claim of nine independent replicates. All lanes share repository state and can influence one another through persistent memory, so statistical analyses must treat them as coupled longitudinal processes.

Every application run is instructed to read current RAKL `main` as the framework source of truth, while new mathematical application work remains in `RAKL_math`. This split serves two purposes. First, problem evidence does not silently become framework authority. Second, a framework defect exposed by one domain can be opened as a separate RAKL issue and evaluated as a method hypothesis rather than patched inside the mathematical result that exposed it.

2.2 Observation is not hidden reasoning disclosure

The observatory does not attempt to reconstruct private model chain-of-thought. The recorded object is an auditable scientific decision trace: problem state, retrieved evidence, proposed action, executed tools, observable result, verification, residual and next action. This distinction is methodologically important. A reproducible research record need not contain the inaccessible or potentially unstable private reasoning process that generated a proposal.

Definition 1 (Task episode)**.** *A task episode is an immutable externally observable record*

$$E = (p, a, c, F, O, \tau, V, y, r, \kappa, t, h),$$

where p is the problem/atom identity, a the active atomic target, c the context identity, F the problem-fibre snapshot, O the operators or methods attempted, τ the public action/observation trace, V the verification evidence, y the outcome, r the residual signature, κ a resource/cost record, t the timestamp and h the content identity.

A non-success outcome should carry an explicit residual or blocker. The raw episode is never replaced by a later reflection summary.

2.3 Episode, diagnosis and lesson are different objects

A central design decision is

$$\boxed{\text{episode} \neq \text{diagnosis} \neq \text{obstruction} \neq \text{lesson}}.$$

An episode records what happened. A diagnosis proposes why it happened. An obstruction generalizes a supported boundary that may apply elsewhere. A lesson proposes a reusable action under explicit triggers and scope conditions. Conflating these stages causes a familiar failure in autonomous agents: a fluent post-hoc explanation is treated as a verified causal rule.

A versioned lesson therefore records

$$L = (g, c, a, e, b, S^+, S^-, f, \alpha, \nu, h_L),$$

with trigger g, scope c, proposed action a, expected effects e, boundaries b, supporting and contradicting episodes S^+, S^-, a falsifier f, authority α, validation obligations ν and content identity h_L. Reusable promotion requires evidence beyond reflection. Raw episodes remain evidence roots even when a lesson becomes useful.

2.4 Problem-conditioned fibres

A research agent should not retrieve the entire archive for every action. For active problem atom A under context c, RAKL compiles a problem-conditioned fibre

$$\mathcal{F}(A \mid P, c) = (K_A, O_A, E_A, B_A, M_A, X_A),$$

where K_A contains epistemic items, O_A applicable tools/operators, E_A analogous episodes, B_A failures and boundary warnings, M_A strategy motifs, and X_A expertise or unresolved warnings. Co-retrieval does not imply compatibility or authority.

The fibre snapshot is important for retrospective method analysis. If a relevant item existed in the bound search universe but was not retrieved, the failure is different from a case in which the item was not present. If the item was retrieved but rejected, the rejection rationale becomes part of the observable decision record.

2.5 Local results and gluing failures

Many difficult research episodes do not fail inside a local lemma. They fail because locally valid pieces cannot be assembled into a root result. RAKL represents this explicitly. A decomposition produces problem atoms with dependencies and optional interface keys. Local sections can be verified independently, but a global solution requires complete coverage, dependency closure and compatibility on shared interfaces.

We therefore distinguish

local mathematical failure

from

gluing/interface failure.

The latter includes missing uniformity across scales, a theorem whose hypotheses are not inherited by the target limit, a representation that does not preserve a root-critical coordinate, or a proof component that is true only in a different category. Failed gluing is itself an experience record and can become a recurring obstruction family.

2.6 Vector saturation

Repeated unsuccessful search should not be summarized as "stuck." The v3 saturation state tracks bounded novelty across seven coordinates:

$$\mathrm{Sat}(R, D) = (S_K, S_O, S_E, S_B, S_R, S_P, S_M),$$

corresponding to knowledge, operator, experience-pattern, obstruction, relation, path and meta-method novelty. An axis is considered bounded-flat only under a registered search window with sufficient independent route coverage, no retained novelty on that axis and no native residual that reopens it. Saturation never licenses an absolute completeness claim.

The observatory records which axis was reopened after a consequential episode. This permits later questions such as whether an agent wastes cycles by continuing to search the knowledge axis when the stable residual is representational or relational.

2.7 Structural novelty of a solved subproblem

A verified local result can be classified by the strongest genuinely new problem-solving structure required:

Class	Interpretation
`STORED`	exact verified solution already registered
`RAKL_TRIVIAL`	composition of existing resources/operators
`TRANSFER_NOVEL`	new application of existing structure with mapping witness
`REPRESENTATION_NOVEL`	new representation/bridge/invariant required
`OPERATOR_NOVEL`	genuinely new transformation/operator required
`ONTOLOGY_NOVEL`	schema/ontology extension required
`UNRESOLVED`	verification or ancestry insufficient

This classification supports an empirical question that cannot be answered from final problem labels alone: what fraction of useful research progress requires genuinely new problem-solving primitives, and what fraction is retrieval, transfer, recomposition, representation repair or gluing?

2.8 Method-case telemetry

For Paper 5, each new consequential research cycle is instructed to emit a bounded `RAKL_METHOD_CASE_STUDY` record. The record contains:

- the research method or operator sequence actually used;
- which prior successes and failures affected routing;
- relevant items retrieved but rejected, and relevant items later discovered to have been missed;
- the chosen falsifier or verification action;
- outcome and residual;

- whether the failure is primarily mathematical, retrieval, decomposition, representation, gluing, source/provenance, verification, tooling/CI or meta-policy;
- the saturation coordinate reopened;
- the strongest defensible novelty class;
- a framework-improvement hypothesis, if one is warranted, with a cheapest prospective discriminator.

The method-case record is an observational layer. It does not authorize a framework change.

2.9 Human and infrastructure interventions

The longitudinal study also records exogenous interventions. Examples include manual schedule-prompt changes, human merge overrides, model/tool upgrades, CI changes and repository migrations. Without an intervention ledger, a later performance change can be falsely attributed to experience learning when it actually came from a stronger base model or a changed tool surface.

Accordingly, a version comparison should bind at least model identity/revision, system-prompt or harness identity, tool policy, resource ceilings, repository state and evaluator identity. If a provider changes an unpinnable model behind an endpoint, comparisons spanning that change are treated as a new observational epoch unless a paired same-time control can recover comparability.

3 Terminology: what "lattice growth" means in RAKL v3

The project name RAKL contains "Knowledge Lattice," but v3 does not assert that all persistent research state forms one global mathematical lattice with a single meet/join operation. The canonical substrate is a typed compatibility and relational structure containing epistemic projections, evidence, operators, episodes, obstructions, strategies and meta-method variants. Lattice/concept-lattice views may be constructed locally where the required order/closure structure is defined and useful.

Accordingly, this paper uses *RAKL state growth* or *retained structured growth* for the quantitative seven-axis process in Section 9. Informal phrases such as "the lattice grows" are shorthand for growth of this typed persistent research state, not a claim that one global lattice cardinality increases monotonically.

This distinction matters empirically. A new obstruction, a new operator and a new epistemic claim are different typed additions and should not be added into one raw scalar. The seven-axis vector preserves these distinctions, while raw node/edge counts remain inventory measurements. A local lattice view can have its own domain-specific size, closure or concept-count statistics, but those statistics are reported with the exact local view and cannot be promoted to a global RAKL complexity measure.

Measurement coverage is equally explicit. The v3 runtime owns episodes, lessons, tools, failures, saturation and optional evolution state. Legacy epistemic `KnowledgeFiber` projections are included in a metrology snapshot only when the caller supplies the exact fibres that define the measurement universe. The resulting unified-substrate hash covers those supplied objects, while the v3 state fingerprint covers only the v3-owned value state. Neither quantity attests arbitrary repository files or external services.

4 Pilot observations from the live research programme

This section reports early qualitative observations from the existing `RAKL_math` research record. The cases were not generated under one uniform v3 telemetry protocol, many were discovered before Paper 5 was defined, and they are not statistically independent. We therefore use them to identify hypotheses and failure classes, not to estimate frequencies or claim causal improvement.

4.1 Retrieval coverage can fail even when memory is correct

The cross-Millennium XM003 audit exposed a particularly informative failure [29]. An earlier synthesis, XM002, stated that Hodge remained an uncalibrated future target for a reusable cross-domain audit operation. XM003 inspected the exact frozen base of that synthesis and found that the Hodge memory review already selected the tool, recorded a target-specific DifferenceWitness, and linked subsequent Hodge calibration/failure artifacts. The evidence existed in the audited state; it was simply not retrieved by the synthesis.

This distinction changes the proposed repair. More memory content would not have helped. The missing object is a coverage certificate that binds the search universe: repository revision, registered lanes/indexes, query coordinates, included and deferred regions, result identities and a re-verification trigger. The application failure therefore produced a framework child issue, RAKL issue 119, asking for a bounded cross-problem coverage receipt rather than another retrieval prompt [30].

The lesson is broader than this instance. A local memory review can be internally consistent while a narrative completeness statement is false because the search universe itself was incomplete. A future RAKL method should distinguish

`NO_RELEVANT_MATCH_IN_BOUND_UNIVERSE`

from an unbounded prose statement that "no relevant memory exists."

4.2 Rich surrogate state can be free

P-versus-NP XM004 tested a source-native closure representation proposed for a cover/fusion route [31, 24]. In the all-rectangle source regime of Cavalar and Oliveira, a singleton generator above an edge has empty trace on the complement; upward closure then forces the entire powerset into the local closure. Raw closure cardinality is therefore maximal even though the source's exact cover-complexity coordinate is zero for that regime.

The mathematical consequence is narrow. It rejects raw closure volume, entropy, or any hardness score that becomes large merely because the generator basis plus upward-closure convention produces many sets. It does not reject the underlying closure representation or more selective pair-indexed productive ancestry.

The research-method consequence is more general. Across hard problems, agents repeatedly construct quantities that are locally meaningful and mathematically clean but are not known to preserve the coordinate required by the root claim. The cheapest adversarial question is often not "can we optimize this quantity?" but

can this surrogate look maximally good while the root coordinate is zero or wrong?

This motivates a reusable `ROOT_COORDINATE_PRESERVATION_AUDIT` candidate operator for later prospective evaluation.

4.3 The same faithfulness pattern appears in unrelated mathematics

The Riemann-Hypothesis analytic lane reached a structurally similar boundary through Li's criterion. Suzuki constructs concrete L^2 objects whose norms are non-negative, but the exact

all-index identity transporting that norm to the Li coefficients is itself RH-equivalent. The resulting atom therefore treats the defect between positive surrogate and target coefficient as the object to audit, rather than treating positivity of the norm as progress toward RH [32, 25].

The RH spectral lane exposed a second representation defect. Under an exact compact-resolvent zero-counting hypothesis, the manuscript context initially suggested ordinary weak-L^1 endpoint behaviour for a first-order resolvent-type operator. An adversarial endpoint calculation found an additional logarithmic divergence and blocked candidate generation before a `CANDIDATE_PROPOSED` event. The frozen packet was preserved and the correction was recorded retrospectively rather than backfilled into the preregistered history [33].

These cases support two different process hypotheses. First, positive or regularized representations need an independently audited faithfulness map to the root quantity. Second, context packets themselves must be falsifiable before candidate generation; a strict chronology is useful only if a defective context can be quarantined without erasing it.

4.4 BSD shows why scalar normalization can hide structural extra zeros

The BSD Selmer/Iwasawa comparison programme asked whether an independently defined discrete or augmentation order can be compared non-circularly with the complex Taylor order at $s = 1$. A source audit of Mazur–Tate/Kato interfaces identified explicit split-multiplicative corrections and examples of extra augmentation zeros, including a good-prime mechanism that can create augmentation vanishing even when Mordell–Weil rank is zero [34, 26].

The route was therefore not repaired by subtracting the desired rank or complex order, which would be circular. Instead the residual asks for independently defined corrections, a no-extra-zero condition, or a richer leading-graded/derived object. For the method case study, this is another instance of representation repair preceding theorem invention.

4.5 A correct local lemma may leave the root essentially untouched

A Yang–Mills spectral candidate provides a clean positive example of local progress [35]. For a positive self-adjoint contraction T, if a dense source set shares one common asymptotic nth-root moment bound $q < 1$, a spectral-projector contradiction implies $\sigma(T) \subseteq [0, q]$. A planted restricted-source example continues to hide a slower mode because the source set is not dense, while a dense source family recovers the true slowest rate.

The result closes an abstract hidden-spectrum logical sub-bridge. It does not establish that the controlled Euclidean Yang–Mills observables map to a dense/cyclic subset of the same Osterwalder–Schrader reconstructed excited Hilbert space, nor that the rate survives support growth, coupling evolution, lattice-spacing limits or continuum spectral identification. Calling the local lemma "progress toward the mass gap" without carrying these interfaces would overstate what was learned.

This case motivates explicit gluing telemetry. A research architecture should be able to report that a local section is supported while the global section remains unavailable because named interface keys are unverified.

4.6 Equation type and source scope can invalidate familiar downstream tools

A recent Navier–Stokes Type-II source route produces an ancient Euler limit in one branch because the rescaled viscosity tends to zero [36, 27]. That observation blocks an otherwise tempting reuse of parabolic Navier–Stokes backward uniqueness as the closing rigidity theorem. The remaining residual concerns no-incoming-energy or far-field tightness inherited by the Euler limit together with an Euler-specific rigidity argument.

This is not evidence that backward uniqueness is a bad method. It is evidence that a method valid in one chart cannot be transported after a load-bearing equation/interface coordinate

changes. In RAKL terms, the failure is a scoped transfer obstruction, not a blacklist of the operator.

4.7 Failing closed on missing source text is a useful research action

The constructive Yang–Mills lane identified Bałaban's dedicated averaging-operations paper as the relevant primary source for an actual weak-coupling block, but the accessible material did not expose enough mathematical body to instantiate the nonlinear averaging map, block/path conventions, covariance equations or regularity hypotheses [37, 28]. The lane left the corresponding contract fields unknown and blocked candidate generation rather than reconstructing a generic block map from memory.

In a conventional benchmark this episode might be scored as no progress. In an open research programme it is a useful narrowing of the evidence boundary. Paper 5 therefore counts source-bound blocking and precise residual creation as legitimate research outcomes while keeping them distinct from theorem progress.

4.8 The v3 merge itself is a governance case study

The framework repository provides a direct meta-level incident. RAKL v3 was merged in commit `a151d561...` with an explicit message that the merge occurred before CI cleanup at operator direction [38]. A later repair PR fixed post-merge CI problems, including the known floating-point assertion in the v3 experience benchmark and unrelated publication/checkout interactions [39].

This incident is useful precisely because it violates the stronger promotion discipline that Paper 5 advocates. A direct user instruction can legitimately override an automated gate operationally; indeed coding-agent instructions are subordinate to direct operator instructions in many harnesses. But an operational override should not retroactively become evidence that the framework change was validated.

We therefore introduce a distinction between *operational deployment state* and *method authority state*. An override can put code on `main` while leaving the method variant provisional for assurance purposes. Post-deployment failures and repairs become immutable episodes. A later promotion-grade claim must bind the exact active state and fresh evidence rather than infer validity from repository location.

4.9 What the pilot does and does not show

The pilot supports a taxonomy of recurring research-process failures that cuts across mathematical domains:

1. retrieval/search-universe coverage;
2. surrogate-to-root faithfulness;
3. representation and correction terms;
4. local-to-global gluing;
5. method-transfer scope and interface changes;
6. source completeness and provenance;
7. chronology/evaluator/CI identity;
8. operational governance versus epistemic promotion.

It does not establish their prevalence, causal effect on success, or uniqueness to RAKL. It also does not show that v3 prevents them, because most episodes were generated before standardized v3 telemetry. Those stronger questions define the prospective evaluation programme in Section 9.10.

5 From raw experience to bounded research method

The purpose of the v3 experience substrate is not to maximize how much past text can be recalled. It is to preserve an auditable relation between what happened, what the system later believes it learned, and what that belief is allowed to change.

5.1 Fast recording and slow consolidation

Every consequential attempt enters the fast loop first:

$$\text{action} \rightarrow \text{observation} \rightarrow \texttt{TaskEpisode}.$$

The episode is frozen before root-cause interpretation. This ordering prevents a later explanation from rewriting the evidence it is supposed to explain.

The slow loop operates on already-recorded episodes:

episodes → contrastive analysis → lesson proposal → replay/transfer validation → scoped method update.

A failed fresh transfer bounds or contradicts a lesson rather than being averaged away. Promotion creates a new lesson version; it does not mutate the source episodes or silently replace the old lesson.

This architecture responds to a known failure mode of agentic memory. Continually rewriting consolidated memory can degrade performance even when the underlying experiences were useful, while retaining raw episodes can remain competitive [9]. RAKL therefore treats consolidation as a defeasible derived view rather than the canonical memory root.

5.2 Authority of a lesson

A lesson can occupy a bounded authority ladder such as

$$\texttt{CANDIDATE} < \texttt{VERIFIED\_LOCAL} < \texttt{CONDITIONALLY\_REUSABLE} < \texttt{PROOF\_BACKED},$$

with `SUPERSEDED` representing historical lineage rather than a higher state. The order is not automatic. A candidate reflection cannot become reusable because the same LLM says it sounds general. A reusable lesson requires independently registered verification and a fresh successful transfer under the relevant implementation contract, or a separately valid proof-backed route.

The current v3 implementation still contains authority-bearing surfaces whose caller-supplied identifiers and flags require stronger content-bound resolution before promotion-grade use. Paper 5 therefore treats new v3 experience in the live mathematical programme as proposal/shadow evidence until these paths are hardened.

5.3 Failure learning is an evidence ladder

Failure learning uses a deliberately slower promotion chain:

`TaskEpisode`(non-success) → observed failure → competing diagnoses → discriminating test → supported diagnosis → boundary lesson candidate → cross-context validation.

The reason is causal. A failed proof attempt may have failed because the theorem is false, because a representation was weak, because the source was incomplete, because a tool was unavailable, because the context packet was wrong, or because CI failed. Treating the first plausible narrative as the causal obstruction would poison future routing.

Diagnosis revisions are append-only versions. A later diagnosis can supersede an earlier one without erasing the original observation. A raw failure cannot directly mint a globally blocking impossibility; only an independently verified impossibility can block reuse, and only inside its registered scope.

5.4 Experience affects routing, not theorem truth

For operator o in target context c, an experience statistic can summarize matched episodes, successes, partial successes, failures and costs. One illustrative routing score is

$$\text{score}(o \mid c) = C(o) + D_V(o) + R_B(o) - \lambda_s \widehat{p}_s(o \mid c) + \lambda_f \widehat{p}_f(o \mid c) - \lambda_e U_o,$$

where C is execution cost, D_V verification debt, R_B boundary risk, $\widehat{p}_s$ and $\widehat{p}_f$ smoothed success and failure/block rates, and U_o a small exploration bonus for undersampled operators. The exact coefficients are a policy choice to be evaluated, not epistemic constants.

The hard rule is structural:

$$\boxed{\text{experience-conditioned priority} \neq \text{epistemic authority}}.$$

A high empirical success rate cannot weaken a proof obligation. A historically poor operator may still be used if the current DifferenceWitness restores a previously broken assumption. This avoids turning experience into a self-reinforcing blacklist.

5.5 Strategy motifs and expertise

Repeated successful contiguous operator sequences can be proposed as strategy motifs. For example, a cross-domain motif may take the form

freeze root coordinate → identify surrogate → construct hostile zero-root control → audit preservation map → rotate representation if the map fails.

The motif is not accepted because it occurred frequently. Contradictory episodes containing the same sequence remain attached to the motif, and applicability is context-indexed.

This separates two kinds of reusable knowledge that recent systems often combine under "skills." Agent Workflow Memory learns reusable routines [5]; AutoSkills constructs multi-level skill hierarchies [6]; and XSkill separates action-level experience from task-level skills [7]. RAKL uses a similar intuition but adds evidence lineage, falsifiers, authority boundaries and cross-context contradiction as first-class fields because the target application is open-ended research rather than only task completion.

5.6 Problem fibres as retrieval state

The problem fibre couples the experience loop to the active research state. For each atom it co-retrieves relevant knowledge, tools, failures, episodes, motifs and expertise. This makes later method analysis possible at a finer level than "memory on/off." A researcher can ask:

- Was a relevant prior failure present in the bound universe?
- Was it retrieved?

- Was it rejected, and on what structural DifferenceWitness?
- Did the chosen operator already have a failure boundary in the target context?
- Did the fibre omit a whole problem lane because the coverage index was stale?
- Did the compiled context exceed the budget and drop a mandatory atom?

These questions convert memory performance into inspectable process variables rather than one opaque retrieval score.

5.7 Learning from gluing failure

Suppose a problem is decomposed into atoms $A_1, \dots, A_n$ and local candidate sections s_i are verified. A global solution requires more than

$$\forall i \ \text{Verified}(s_i).$$

The sections must cover all required atoms and agree on shared interfaces. Let I_{ij} denote the set of declared shared interface coordinates between A_i and A_j. Then a simplified compatibility condition is

$$\forall x \in I_{ij}, \qquad s_i(x) = s_j(x),$$

plus dependency and authority conditions.

In research, the interface may be a uniform constant, a limit theorem, a category-preserving realization map, a source-density result, or a statement that a local invariant actually controls the root quantity. A gluing obstruction therefore has a typed residual signature rather than being summarized as "proof incomplete."

The pilot cases suggest that gluing failures may be unusually common in hard open problems. Yang–Mills exhibits fixed-cutoff versus continuum and source-to-spectrum interfaces; BSD exhibits discrete/p-adic versus complex analytic coordinates; Hodge exhibits cohomological detection versus algebraic realization; Navier–Stokes exhibits compactness/limit-class versus rigidity hypotheses. Paper 5 preregisters gluing-failure detection as a process QoI because earlier recognition may save large amounts of unproductive local theorem search.

5.8 Saturation and the invention gate

The system should not infer "missing operator" merely because repeated attempts fail. Invention is proposed only after a bounded saturation audit supports all of the following:

1. relevant knowledge/operator/path axes are flat under the registered search window;
2. the residual is stable across repeated independent routes;
3. ordinary causes such as missing source detail, wrong context, unavailable tool, implementation failure or verification debt have been excluded;
4. cross-domain transfer routes have been searched or bounded;
5. explicit evidence supports a representation or method-basis gap.

The resulting report permits escalation to an invention process. It does not prove that invention is necessary. This fail-closed distinction is important for studying "scientific creativity." A system that invents new formal objects whenever it is confused may look creative while simply failing to retrieve or compose what already exists.

5.9 RAKL-triviality as an empirical hypothesis

Let $N_R(P)$ denote the minimum genuinely novel problem-solving structure required for a verified solution of problem P relative to RAKL state R. If existing resources, operators and motifs suffice through composition or transfer, then $N_R(P) = 0$. For a solved distribution D we can define

$$\rho_R(D) = \frac{|\{P \in D : N_R(P) = 0\}|}{|D|}.$$

This is not assumed to be high. It is a measurable hypothesis about the relationship between the growth of reusable method space and the growth of problem space.

The longitudinal observatory extends the question from terminal solutions to research progress. A problem may remain unsolved while a useful subproblem is classified as transfer-novel or representation-novel. Over many episodes, the distribution of novelty classes can reveal whether the system's bottleneck is missing knowledge, missing representations, missing operators or merely poor routing among already-known components.

5.10 Unified substrate and Self-RAKL

The v3 unified substrate materializes a read-only typed overlay over specialized stores. Epistemic items, operators, episodes, failures, strategy motifs and architecture variants can be related without transferring semantic authority to the overlay itself. This lets Self-RAKL represent its own variants as meta-method objects while preserving the original stores that own proof, failure or promotion semantics.

The intended recursion is therefore not

LLM rewrites itself and declares success.

It is

observed research → bounded method hypothesis → challenger variant → matched evaluation → fresh assurance → governed promotion or rejection.

The next sections make this engineering loop explicit.

6 Implementation status at the Paper 5 evidence cutoff

Because this paper combines deployed v3 machinery, integrated measurement instrumentation and prospective evaluation, Table 3 states the implementation boundary explicitly. The source-level matrix is maintained in `docs/PAPER5_IMPLEMENTATION_STATUS.md`. "Deployed" or "implemented" never means that a fresh-task capability improvement has been demonstrated.

Table 3: Status of major Paper 5 surfaces. Implementation, deployment, internal hardening, empirical improvement and independent assurance are distinct coordinates.

Surface	Status	Licensed statement
TaskEpisode/Lesson substrate, failure revisions, problem fibres, saturation, novelty classes, unified substrate	deployed implemented	v3 can persist, relate and query typed research experience; this alone does not establish fresh-task improvement

Surface	Status	Licensed statement
Experience-conditioned routing and strategy motifs	deployed implemented	experience can alter search ordering while hard epistemic gates remain separate; benefit remains empirical
Protected lesson/tool promotion	deployed, internally hardened	authority-bearing promotion resolves protected content-bound evidence rather than relying only on caller narratives; independent external assurance is not claimed
Certificate-backed local-section/gluing authority	deployed, internally hardened	local verification and global gluing authority are subject-bound and fail closed under the implemented protected path; mathematical correctness still depends on the registered evidence/checker boundary
Evolution archive and governed promotion binding	deployed, internally hardened	variants can branch while preserving rollback targets; governance/evolution claims are bound to protected subjects rather than a bare Boolean on the current path
Experience benchmark subject binding	deployed, internally hardened	protocol and result identities can be content-bound for matched experience evaluation; benchmark gain still grants no global capability claim
Paper 5 state/process metrology	integrated measurement-only	read-only quantitative snapshots, seven-axis retained-growth accounting, process telemetry and cost-policy identity can be recorded; metrology has no promotion authority
Four-arm causal-attribution objects and scripts	integrated, preregistered; unexecuted confirmatory study	MODEL_ONLY, RESET, SHAM, LEARNING comparisons and state-leakage checks are executable; no causal performance result is reported
Method-state manifest and upgrade protocol	integrated governance map	coding agents can identify the incumbent method epoch, classify changes and construct challengers; the manifest itself grants neither promotion nor evolution evidence
Cross-problem memory coverage receipt	open framework problem	issue #119 identifies a coverage gap; unbounded "no relevant memory" claims remain unlicensed
Independent retained-novelty audit	protocol frozen, evidence pending	internal seven-axis novelty accounting is measurable but is not represented as externally validated semantic novelty
Independent external assurance of v3 hardening	pending	current implementation closes known declaration-bound authority paths under internal tests; independent/process-lineage-separated assurance remains a stronger unperformed claim

Surface	Status	Licensed statement
RAKL 3.1 superiority or incumbent status	not established	a future Class-B challenger must earn that label through matched development, fresh protected assurance and governance

This separation is itself part of the method. A new commit can change deployment or implementation status, but it cannot automatically convert a prospective empirical claim into an evidence-supported capability statement. Conversely, a negative or null four-arm result would remain a valid Paper 5 result rather than being hidden by further framework edits.

7 A governed upgrade constitution for recursive research systems

The central engineering risk in self-improvement is semantic collapse between four distinct events:

code exists $\neq$ code is deployed $\neq$ improvement is supported $\neq$ the method is the governed incumbent.

A software branch can be implemented and even merged for operational reasons while its scientific method claim remains unvalidated. The live v3 deployment supplies an instructive example: the v3 branch was merged into `main` under an explicit operator request before its pending CI had completed, and later commits repaired package/publication CI. That sequence is preserved as process history. It is not represented as evidence that v3 was superior merely because it was deployed.

7.1 Method identity is not the package version

The inspected Python package still reports software version `0.1.0`. We therefore separate software/API compatibility from research-method identity. A complete experimental citation should bind at least

$$V = (v_{\text{pkg}}, v_{\text{method}}, e_{\text{constitution}}, \Sigma_{\text{schemas}}, h_{\text{git}}, h_{\text{validators}}).$$

We use `3.0.0` as the method identity for the deployed v3 architecture at the Paper 5 freeze, while the exact Git subject remains the decisive reproducibility identity. Patch-level method versions such as 3.0.x are reserved for implementation corrections that do not intentionally change research policy. Minor method versions such as 3.1 and 3.2 denote workflow/method challengers. A change to protected authority/governance semantics is constitutional and should advance a separately visible constitution epoch or major method generation rather than being hidden in an ordinary workflow patch.

The Paper 5 branch introduces a proposal-only `RAKL_VERSION.json` manifest to make these coordinates machine visible. Its presence does not promote itself to canonical governance.

7.2 Change classes

We retain the existing three-way classification in `meta.py`.

Class A: implementation. Examples include parser defects, CI portability, serialization, hashing, deterministic replay and publication build repairs. A Class-A change requires a reproducing regression, relevant invariant tests, artifact identity and preservation of history. It can improve engineering reliability without supporting a method-capability claim.

Class B: workflow or method. Examples include retrieval policy, fibre compilation, routing, saturation, gluing, lesson induction, memory consolidation and research-portfolio allocation. A Class-B challenger must state a positive meta-QoI prediction before evaluated outcomes, run matched parent/challenger development tests, include fresh transfer and hostile near-misses, preserve blocking invariants, use comparable resources, and pass protected fresh assurance before a strong evolution claim.

Class C: constitution. Examples include changing what evidence may mint authority, weakening review independence, changing the meaning of missing evidence, or changing who can promote an incumbent. Constitutional changes are never auto-promoted. They require explicit human-visible amendment review, migration/rollback analysis and adversarial authority tests.

7.3 Roles and anti-self-certification

A strong evolution claim separates at least seven roles, even if one organization operates them:

1. **observer** records content-bound research episodes and process telemetry;
2. **upgrade proposer** maps repeated evidence to a falsifiable framework hypothesis;
3. **challenger engineer** implements the frozen proposal;
4. **development evaluator** runs visible debugging/replay tests;
5. **fresh assurance evaluator** uses a packet frozen before mutation and hidden from the optimizer;
6. **governance authority** decides whether an assured variant becomes incumbent;
7. **post-promotion attestor** observes the actual active repository state and exact-main validation.

Same-session role labels do not create independence. Process independence and evidence-lineage independence must be separately established when independent credit is claimed.

Proposition 1 (No self-promotion from proposal output)**.** *Suppose a challenger cannot modify the protected evaluator, assurance packet, promotion thresholds or governance credentials that judge it, and promotion requires a content-bound certificate issued outside the challenger path. Then the challenger's generated narrative or code output alone is insufficient to establish its own promotion.*

Proof. The promotion predicate contains at least one required input whose value is controlled outside the challenger and is bound to the exact challenger identity. Changing the candidate output cannot set that external input to an accepting value. Therefore self-generated output is insufficient for promotion, even if it can affect development metrics. The claim is conditional on the protection and identity assumptions; compromise of those boundaries remains a separate threat model. □

7.4 Upgrade proposal packet

Before implementation outcomes are observed, a Class-B/C proposal freezes a machine-readable packet containing the parent method/Git identity, source episode and diagnosis IDs, change class, alternative diagnoses, proposed mechanism, affected method contracts, predicted primary and secondary meta-QoIs, material-effect thresholds, possible regressions, negative controls, hostile

near-misses, development benchmark, fresh assurance reserve, model/tool/resource contract and rollback plan. If the result predates the packet, it is retrospective calibration only.

This packet is intentionally stricter than an issue description. It converts the statement "we think this might help" into a falsifiable prediction about a content-identified framework mutation.

7.5 Challenger lifecycle and variant DAG

We model evolution as a DAG rather than a destructive sequence:

$$\texttt{OBSERVED\_PATHOLOGY} \to \texttt{UPGRADE\_HYPOTHESIS} \to \texttt{PREREGISTERED} \to \texttt{CHALLENGER\_IMPLEMENTED}$$
$$\to \texttt{DEVELOPMENT\_VALIDATED} \to \texttt{FRESH\_ASSURANCE\_PENDING} \to \{\texttt{ASSURED}, \texttt{REJECTED}, \texttt{META\_OVERFIT}, \texttt{CANNOT\_CHECK}\}$$
$$\to \texttt{GOVERNANCE\_APPROVED} \to \texttt{PROMOTED} \to \texttt{ACTIVE\_PROMOTION\_ATTESTED} \to \texttt{MONITORED}.$$

Rejected and superseded variants remain negative history. A previous incumbent remains a rollback target when compatible.

The existing v3 `EvolutionArchive` already separates challenger, assured, rejected and incumbent states and does not auto-promote a challenger merely because an evolution trial passes. However, its current `promote_incumbent` interface accepts a caller-provided `governance_approved` Boolean. We regard that as an implementation gap, not as adequate promotion evidence. A future 3.1 challenger should replace declaration-bound governance with a content-bound attestation naming exact candidate, assurance verdict, constitution epoch and approving process.

7.6 Development evaluation versus fresh assurance

Two evidence phases answer different questions. Visible development evaluation can guide debugging and optimization; once exposed, it is no longer blind assurance. Strong method-evolution evidence therefore consumes a separate fresh packet frozen before the challenger and hidden from the proposer. Repeated peeking consumes the assurance exposure budget. The existing `SelfEvolutionAssessor` and `evaluate_bootstrap_trial` already distinguish development gain, fresh transfer, evidence-lineage independence, evaluator separation, resource matching and meta-overfit. These components support a scoped verdict rather than a global "RAKL evolved" state.

7.7 Evaluator migration

A reviewer may reasonably object that fixed evaluators eventually become obsolete. We allow evaluator evolution, but not inside the candidate it judges. A legitimate evaluator change is a separately governed parent mutation. Within an evaluation epoch, the ruler is frozen. At epoch boundaries, a new evaluator can be proposed, benchmarked against known-answer and adversarial cases, versioned and then used prospectively. This differs from allowing the optimizer to rewrite its reward after seeing an inconvenient score.

7.8 Operator override and deployment before assurance

Human or repository governance can intentionally override the scientific promotion process. The protocol therefore models an explicit `OPERATOR_OVERRIDE` with requested deviation, exact subject, blockers known at the time and containment/rollback plan. An override may move code if repository permissions allow, but creates no evolution-evidence credit. The deployed variant remains method-provisional until the required evidence is produced. This rule is not hypothetical; it is motivated by the actual v3 merge-before-CI incident and prevents the paper from retrospectively rewriting operational history into a clean preregistered story.

7.9 Post-promotion attestation and rollback

Pre-promotion authorization is not evidence that deployment occurred correctly. The existing `promotion_attestation.py` distinguishes a candidate that passed checks from an active state whose `main` ref descends from the candidate, required content matches and exact-active-main post-promotion validation succeeds. A promoted variant also carries an exact rollback parent, migration notes, rollback test and monitoring conditions. New evidence can narrow, supersede or roll back a variant without erasing the evidence that originally supported it.

7.10 Version increments as empirical claims

Under this constitution, a label such as `3.1` has scientific content. It means that a distinct workflow/method challenger has been identified and evaluated under the registered process; it does not mean that a certain number of commits were merged. Consequently Paper 5 can plot RAKL 3.0, 3.1 and later versions as experimental interventions rather than arbitrary software milestones.

8 Informing coding agents: repository-native method state and Codex harness

Recursive framework governance is ineffective if the coding agent modifying the package does not know which method version is incumbent, which rules are protected, or how a proposed change will be evaluated. Conversely, putting every historical fact into a giant system prompt consumes context and quickly becomes stale. We therefore use the repository itself as the durable system of record and make the coding-agent entrypoint a navigation layer.

8.1 Instruction hierarchy

OpenAI's Codex documentation specifies that repository `AGENTS.md` files can provide coding conventions, repository structure and testing instructions; their scope follows the directory tree, more deeply nested files can specialize broader instructions, and direct system/developer/user instructions take precedence [21]. Codex's published harness-engineering guidance further argues for giving the agent a map rather than a very large instruction manual, with deeper repository documentation serving as the source of truth [22]. We adopt that hierarchy rather than assuming the agent carries a persistent private memory of the latest RAKL architecture.

The Paper 5 challenger adds a compact self-evolution entrypoint to the existing root `AGENTS.md`. For a task that can change framework behavior it points to:

1. `RAKL_VERSION.json`, a machine-readable method-state manifest;
2. `docs/RAKL_UPGRADE_PROTOCOL.md`, the governed challenger lifecycle;
3. `docs/RAKL_METROLOGY.md`, the frozen measurement vocabulary;
4. `src/rakl/method_specs.py`, the canonical process contracts;
5. protected promotion/evolution/attestation surfaces relevant to the requested change.

The intent is not to make `AGENTS.md` the constitution itself. A future engineering cleanup should further reduce the root file toward a table-of-contents role while preserving exact deeper sources of truth.

8.2 Startup protocol for a framework-editing Codex session

Before editing a framework method, a coding agent should execute the following observable startup sequence:

1. resolve the current exact `main` Git SHA and method-state manifest;
2. classify the request as Class A, B or C;
3. locate the affected method contract and protected evaluator/authority surfaces;
4. inspect the relevant case-study episodes, diagnoses or lessons motivating the change;
5. freeze the upgrade proposal, target metrics, benchmark/evaluator identities, negative controls and rollback plan when Class B/C;
6. create or use a challenger branch rather than silently rewriting the incumbent;
7. implement the smallest change that tests the stated mechanism;
8. run exact-subject tests and emit a machine-readable handoff containing parent, candidate, metrics, blockers and rollback identity.

This sequence is intentionally inspectable from repository artifacts. We do not require disclosure of hidden model chain-of-thought.

8.3 What happens when a user directly instructs Codex to bypass the protocol?

The instruction hierarchy means a direct user/operator request can override repository guidance [21]. The method therefore cannot rely on `AGENTS.md` as an access-control mechanism. The correct response is architectural rather than rhetorical. The repository records an operational override and withholds evolution-evidence credit. Protected CI, branch protection, external assurance and post-promotion attestation remain separate controls where configured. Thus the scientific statement is not "Codex cannot disobey the method"; it is "an override is observable and does not become evidence of method improvement merely because the code moved."

8.4 Method-state manifest

The proposal-only `RAKL_VERSION.json` introduced on the Paper 5 branch names the current method generation, exact Git subject at proposal freeze, software package version, constitution epoch, protected evaluation surfaces and known upgrade gaps. A canonical future form should also bind active schema versions, evaluator bundle hashes, migration status and last attested promotion.

The manifest solves a practical coordination problem. A newly started Codex session, human developer or other coding agent can determine the intended incumbent without relying on conversational memory. At the same time, the exact Git SHA prevents a friendly version label such as "3.1" from hiding divergent code.

8.5 Package/API version versus method version

Software consumers need ordinary API compatibility semantics, whereas scientific evaluation needs method identity. These are different concerns. A serialization bug might require a Python package patch without changing research behavior. Conversely, a routing-policy change might constitute a new research method even if public Python APIs remain backward compatible. The manifest therefore records both. Release artifacts should additionally use the repository's

content-addressed release manifest so that binaries or generated documents can be tied to an exact source subject.

8.6 State and schema migration

Persistent experience means framework upgrades cannot treat state migration as an implementation footnote. A new version must state whether old episodes, lessons, failures, tools and saturation records are:

- byte-compatible and directly readable;
- translated by a content-bound deterministic migration;
- retained only as legacy read-only evidence;
- or incompatible and therefore excluded from the challenger evaluation.

A migration must never manufacture higher authority. If a new schema can express stronger authority than the old one, imported objects retain the old effective authority until independently revalidated. Migration tests include round-trip identity where promised, source-pin preservation, supersession lineage and rollback compatibility.

8.7 Concurrent agents and branch isolation

Long-running research may involve several solution agents, a meta-observer and one or more coding agents. Concurrent work creates race conditions in both software and epistemic state. The safe default is immutable source episodes plus isolated challenger branches. A framework engineer should rebase or merge current `main` before final evaluation, recompute exact subject hashes and rerun candidate checks. A benchmark packet bound to an earlier head cannot be silently claimed for a later integrated head.

The same principle applies to mathematical applications. `RAKL_math` remains the application-state repository; reusable framework mutations belong in `RAKL`. Application evidence may motivate a framework issue but cannot write itself directly into framework authority.

8.8 Reproducibility under model-service drift

Repository state can be exactly versioned; hosted model behavior may not be perfectly reproducible even under the same public model name. Strong experiments therefore record model identity/revision when available, prompt hashes, temperature, seed where supported, token ceilings, tool policy, external retrieval policy and wall-time/resources. If an exact model revision is unavailable later, the replication is a new condition rather than an exact rerun. Framework evolution claims should be robust enough to survive matched tests across more than one model/revision when the claim is intended to generalize beyond a specific model.

8.9 Why the coding agent is part of the experiment

The coding agent is not merely infrastructure. Its engineering behavior supplies additional Self-RAKL case evidence: evaluator drift, CI repair, migration defects, stale instructions, over-large context files, unsafe authority interfaces and rollback friction are all observable framework-process outcomes. Paper 5 therefore treats engineering incidents as `TaskEpisode`-like evidence at the meta-method level rather than cleaning them from the historical record once fixed.

9 Quantifying growth, process quality and causal assistance

A recursive research architecture is not supported by the observation that its repository grows or that its underlying language model eventually succeeds. A system can accumulate duplicated or irrelevant memory, spend more compute, and still attribute the model's unaided ability to the framework. We therefore treat metrology as part of the method rather than as post hoc visualization. The measurement contract is implemented prospectively in `docs/RAKL_METROLOGY.md` and the read-only `src/rakl/v3_metrology.py` surface on the Paper 5 challenger branch. These measurements grant no scientific or promotion authority.

9.1 Seven-axis retained-growth vector

The state-growth coordinates reuse the v3 saturation axes. For research cycle t we define

$$g_t = (\Delta K_t, \Delta O_t, \Delta E_t, \Delta B_t, \Delta R_t, \Delta P_t, \Delta M_t),$$

where the components denote retained novelty in knowledge, operators, experience patterns, obstructions, relations, successful paths and meta-methods, respectively. The qualifier *retained* is essential. Raw files, commits, nodes or generated lessons are inventory, not evidence of learning. The v3 `NoveltyRound` contract records retained semantic novelty and separates it from repeated route searches or paper count.

Alongside g_t, each frozen state reports node counts by substrate kind, edge counts by typed relation, episode outcomes, lesson and tool authority distributions, failure-diagnosis status, evolution-variant status and unresolved cross-view links. For states S_t and S_{t+1}, `compare_state_metrics` produces a content-independent growth delta. We do not collapse these quantities into a single "lattice size" score.

Several ratios diagnose whether growth is useful rather than merely large. We preregister lineage coverage as the fraction of derived canonical objects with valid source ancestry, unresolved-link rate in the common substrate, supersession-preservation rate, and a semantic novelty-retention ratio that compares retained additions with raw proposals once the necessary proposal identity stream is available. High raw growth with low lineage coverage or high orphan rate is a negative result, not progress.

9.2 Experience conversion and failure learning

The fast v3 loop stores immutable `TaskEpisode` records; the slow loop may propose lessons, diagnoses, tools and strategy motifs. We measure the conversion funnel rather than assuming that every stored episode is useful:

$$\text{episodes} \to \text{supported diagnoses or lessons} \to \text{reusable tools or motifs} \to \text{validated fresh reuse}.$$

Reported quantities include episode outcome rates and costs, lesson proposal and validation yields, tool reuse success and failure rates, failure-diagnosis maturation, time from observation to supported diagnosis, obstruction-resolution rate, and the repeated-failure rate

$$R_F = \frac{\#\{\text{failures with a previously observed structural signature}\}}{\#\{\text{failures}\}}.$$

A useful experience system should reduce R_F on fresh tasks without increasing false transfer. Neither a high lesson yield nor a low lesson yield is intrinsically desirable. Excessive abstraction can be as harmful as failure to learn.

9.3 Process telemetry for the full method surface

RAKL already declares canonical process surfaces in `method_specs.py`. Paper 5 requires consequential invocations to be measurable. A proposal-only `ProcessTelemetry` record binds the method surface, input and output state identities, active task and fibre, outcome, registered cost, residual transformation, retrieved/selected/rejected object identities, verification artifacts and the retained novelty vector. The common aggregate for process p reports

$$N_p, \quad P_p(\text{success}), \quad P_p(\text{blocked}), \quad P_p(\texttt{CANNOT\_CHECK}), \quad \overline{C}_p, \quad \overline{\Delta|B|}_p,$$

plus axis-specific retained novelty and object-selection counts. Here B denotes the registered residual set; raw residual-count contraction is descriptive because replacing one vague blocker by several precise obligations can represent epistemic progress despite increasing $|B|$.

The process-specific dashboard covers decomposition, routing, search-query generation, source qualification, claim extraction, ontology normalization, context translation, equivalence/structural transfer, contextual gluing, contradiction diagnosis, gap discovery, discriminator selection, synthesis, memory, review, benchmarking, authority promotion, saturation stopping, context compilation, capability shaping, software execution, research-portfolio allocation, objective evolution and generator transport. The complete definitions are frozen in the metrology contract. Representative quantities include atom reopen rate and late interface discovery for decomposition; saturated-route retry rate for routing; retrieval precision and recall in a bound universe for memory; hostile-near-miss rejection and false-transfer rate for transfer; local-success/global-failure rate for gluing; pre-promotion defect capture for review; exact-subject replay for software execution; and invalid promotion rate, whose target is zero.

9.4 Residual contraction as a common progress coordinate

Open research often lacks a scalar objective before the target theorem or artifact exists. We therefore record progress through typed residual transformations. Let B_t be the canonical active blocker set for an atom. A simple contraction

$$C_t = |B_t| - |B_{t+1}|$$

is reported only when blocker identity is stable. The more informative representation records blockers resolved, newly exposed, unchanged and reopened. This prevents the framework from manufacturing "progress" by renaming an obstruction or hiding a root-like obligation inside a new representation.

9.5 Four-arm causal-attribution benchmark

The key empirical question is whether RAKL contributes anything beyond the base model. We therefore extend the earlier reset-versus-learning design with a separate four-arm attribution benchmark. The same frozen task packet, model revision and registered resource ceiling are used in every arm:

`MODEL_ONLY` The same base LLM and permitted external tools operate without the RAKL workflow or persistent RAKL state.

`RAKL_RESET` The RAKL workflow is active, but every evaluation task starts from the same initial framework state. This estimates the static architecture effect.

`RAKL_SHAM_MEMORY` The workflow and memory/context budget are matched, but relevant learned memory is replaced with registered irrelevant or structurally incompatible controls. This tests whether gains arise from experience content rather than merely extra context or preprocessing.

`RAKL_LEARNING` The full workflow uses the learned state produced by the development sequence. Every fresh-transfer task starts independently from the same frozen learned state.

For registered metric m we report separate lift vectors

$$\Delta_{\text{architecture}}(m) = m(\texttt{RAKL\_RESET}) - m(\texttt{MODEL\_ONLY}),$$

$$\Delta_{\text{experience}}(m) = m(\texttt{RAKL\_LEARNING}) - m(\texttt{RAKL\_RESET}),$$

$$\Delta_{\text{content}}(m) = m(\texttt{RAKL\_LEARNING}) - m(\texttt{RAKL\_SHAM\_MEMORY}),$$

and

$$\Delta_{\text{total}}(m) = m(\texttt{RAKL\_LEARNING}) - m(\texttt{MODEL\_ONLY}).$$

There is deliberately no scalar "RAKL score".

For binary task success, each matched pair is also assigned to `BOTH_SUCCESS`, `RAKL_ONLY_SUCCESS`, `BASELINE_ONLY_SUCCESS` or `BOTH_FAIL`. The asymmetric counts are scientifically important. A `RAKL_ONLY_SUCCESS` is direct scoped evidence of assistance; a `BASELINE_ONLY_SUCCESS` is direct evidence that the framework interfered and must be reported with equal prominence.

9.6 Efficiency and epistemic-safety lift

Framework contribution is not limited to final success. When both arms solve a task we compare model tokens, preprocessing tokens, retrieval calls, tool calls, wall time, candidates attempted, failed route families and time to the first decisive falsifier. A framework that spends substantially more resources for the same result is not credited as a capability gain unless the claim is explicitly about a quantified quality or safety tradeoff.

For open research, epistemic-safety effects may appear earlier than root solutions. We therefore measure unsupported scope escalation, false theorem/candidate promotion, invalid structural transfer, surrogate-to-root coordinate errors, false global gluing, chronology violations and source-scope errors. A framework that lowers false progress while holding solution rate fixed may still be scientifically useful, but that claim is kept separate from capability improvement.

9.7 Decision-level contribution witnesses and component ablation

Population-level matched trials establish the main attribution. To understand mechanism, a consequential intervention may emit a proposal-only contribution witness recording what RAKL consulted, how route ranking changed and what action was selected. Examples include failure memory demoting a repeated route, a retrieved tool promoting a cross-domain method, a root-coordinate preservation gate blocking a seductive surrogate, or a gluing check exposing a missing interface.

Such a witness is not causal proof. Where feasible, we preregister component ablations or leave-one-memory-out replay. Candidate ablations include removing failure memory, success-tool memory, experience-conditioned routing, problem-fibre compilation, gluing checks, saturation/invention gates, or a specific high-impact lesson. Repeated seeds or sampling configurations are required before interpreting stochastic replay differences as stable effects.

9.8 Statistical units, dependence and uncertainty

Tokens and tool calls are not independent observations. The primary unit is the task for matched outcome comparisons, the episode for process/failure measurements, the problem family or domain for transfer generalization, and the framework version for evolution claims. Development episodes are temporally dependent by design because the state changes. Fresh-transfer tasks must all begin from the same frozen learned state so that transfer task T_1 cannot train T_2.

We use paired analyses whenever the same task can be evaluated under multiple arms. Binary success is reported through paired contingency counts and an appropriate paired interval/test. Continuous scores and resource quantities are analysed as paired differences with bootstrap, permutation or model-based uncertainty appropriate to the frozen sampling design. If sufficient task families accumulate, hierarchical analyses will separate within-family from cross-family effects. Primary and secondary metrics, material-effect thresholds, multiplicity policy, resource ceilings, stopping rules and assurance exposure budgets are frozen before evaluated results.

9.9 Goodhart resistance and non-compensatory invariants

Once a metric influences upgrades it becomes a target. We therefore prohibit one-number promotion. Benchmark and evaluator hashes, thresholds and hard authority invariants are protected from the challenger they judge. Raw storage growth never counts as improvement, hostile near-misses expose over-transfer, and a gain in success or efficiency cannot compensate for an authority, history, evaluator-integrity or fresh-transfer leakage failure. If an upgrade legitimately changes a metric or evaluator definition, that evaluator migration is treated as a separately governed parent change rather than being allowed to redefine the ruler inside the challenger.

9.10 What would demonstrate that RAKL is doing real work?

We preregister a minimum evidentiary chain for the statement that the framework learned or assisted:

Repository size, issue count, prose volume, proposed lesson count, or an agent's self-report that RAKL was helpful do not satisfy this chain. The strongest version-level claim additionally requires protected fresh assurance and governed promotion under the upgrade protocol in Section 7.

10 Security, governance and failure containment

A self-modifying research framework is also an adversarial system. The relevant adversary need not be a malicious external actor. An optimizer that legitimately seeks a higher score can accidentally exploit stale evaluators, contaminated memory, ambiguous identities or permissive authority interfaces. We therefore treat the model, retrieved text and challenger code as untrusted proposal sources with respect to protected authority.

10.1 Trusted computing base and its boundary

The intended trusted computing base is deliberately smaller than the research agent. It includes the protected Constitution/authority rules, exact-subject identity checks, evaluator and assurance packet identities, promotion/attestation logic, and the repository or execution mechanisms used

to observe these facts. The language model, same-session reviewer roles, generated lessons, retrieved documents and challenger implementation are not trusted to certify themselves.

Content hashes establish identity under the assumed hash function; they do not establish semantic truth. A Git commit proves neither that the mathematical claim is correct nor that the evaluator is well designed. Likewise, a signed governance attestation would establish who approved an exact object, not that the approval was scientifically wise.

10.2 Memory poisoning and prompt injection

Persistent experience creates an attack surface absent from stateless agents. A malicious or simply erroneous episode can be repeatedly retrieved and gain influence through consolidation. Retrieved webpages, papers, issues and code comments may contain imperative language that attempts to act as instructions. RAKL therefore separates evidence content from agent-control instructions. External content can support claims or propose actions, but cannot alter the Constitution or coding-agent instruction hierarchy merely by being retrieved.

Derived lessons preserve supporting and contradicting episode lineage. A lesson is not allowed to delete the raw evidence from which it was distilled. This design is motivated both by general provenance principles and by recent empirical work showing that repeated LLM memory consolidation can degrade useful memories even when the underlying episodes remain informative. A later contradicted or superseded lesson remains inspectable rather than overwriting the episodes that expose the error.

Hostile memory tests should include fabricated high-confidence lessons, duplicated episodes, contradictory success/failure histories, stale-context reuse, malicious instruction text embedded in evidence, and a lesson whose apparent support consists of many copies from one lineage.

10.3 Evidence Sybils and pseudo-independence

Counting artifacts creates an obvious Sybil vulnerability. Ten summaries of one source do not constitute ten independent sources, and ten role labels in one context do not constitute ten independent reviews. Metrics therefore count source/evidence lineage and process independence rather than raw citation or agent count. Repeated failures derived from one copied episode cannot manufacture a cross-context obstruction pattern; repeated successes from the same lineage cannot manufacture transfer breadth.

10.4 Evaluator hacking and Goodhart pressure

An evolving agent will eventually discover regularities in any visible benchmark. The protection strategy is layered:

- development tasks are explicitly reusable and may be optimized against;
- strong assurance tasks are frozen before the challenger and hidden from the proposer;
- assurance exposure is budgeted, and exposure converts a packet into development evidence;
- evaluator code, thresholds and subject-identity rules are protected from the challenger they judge;
- hostile near-miss tasks measure over-generalization;
- no scalar metric can compensate for an authority or integrity failure.

A legitimate evaluator evolution is possible only through a separate governed change between evaluation epochs. This matters because recent self-improving-agent research has begun to

explore co-evolving evaluators. Our design does not assert that a fixed evaluator should remain optimal indefinitely; it asserts only that the candidate must not rewrite its own ruler within the trial used to claim improvement.

10.5 Authority forgery and protected subject binding

The first v3 integration audit identified a concrete defect class: caller-provided booleans and opaque string identifiers could be mistaken for high-authority evidence. The current implementation has since hardened the known lesson/tool, local-section/gluing, evolution/governance and benchmark authority paths with protected content-bound subject hashes, attestation resolution and fail-closed bindings. Thus the historical defect is retained as negative engineering evidence rather than presented as a property of the current code.

The governing invariant is

$$\text{authority transition} \Rightarrow \text{resolved exact subject} + \text{licensed evidence type} + \text{protected gate} + \text{scope compatibility}.$$

Internal conformance and hostile tests can show that the implemented path enforces this contract for registered attacks. They do not prove that the trust roots themselves are uncompromised, that every future call site is correct, or that an external evaluator would reproduce the assurance claim. Residual risks therefore include stale or misbound registry entries, protected-key compromise, evaluator contamination, operator override and ordinary implementation defects. Paper 5 distinguishes implementation hardening from independent external assurance.

10.6 Chronology attacks and backfilling

If an agent first observes a result and then creates a "preregistered" packet that predicts it, the evaluation is contaminated even when every file is internally consistent. Hash-chained traces and frozen packet identities make this harder to hide, but only if the upstream chronology is actually observed. Retrospective results are useful calibration and learning evidence; they simply receive zero prospective credit.

The live case study contains several chronology/provenance repairs, including research results that were downgraded to retrospective calibration rather than being backfilled into a strict discovery trace. We treat this as a positive governance behavior and a process-cost signal at the same time.

10.7 Cross-task leakage

Persistent learning is supposed to transfer development experience to fresh tasks, but transfer tasks must not teach one another inside the same assurance packet. The v3 benchmark therefore freezes the learned state after development and starts every fresh-transfer task independently from that exact state. A run whose initial state already contains a prior transfer result invalidates the packet.

The same principle applies across benchmark arms. The sham-memory condition must be constructed before target outcomes and must not accidentally contain semantically equivalent answers or target-specific hints.

10.8 Supply chain, external tools and model services

RAKL depends on Python libraries, Git hosting, CI runners, external search services and hosted models. Exact repository hashes do not pin all of these dependencies. Strong release/evaluation receipts should therefore record environment and dependency identities where material, and distinguish an unavailable dependency from a scientific refutation. A supply-chain compromise

or hosted-model behavior change can invalidate reproducibility without falsifying the method hypothesis.

10.9 Denial of service through memory growth

A persistent system can fail by becoming too large to query. Storage growth is separated from prompt growth; problem fibres and context compilation create bounded working sets. Metrology records useful-context fraction, retrieval miss rate and context size. Compaction or pruning may create derived views, but raw canonical episodes remain evidence roots unless a separately governed retention policy applies. Resource stress tests should include memory stores large enough that naive full-context loading is impossible.

10.10 Governance attacks and operator powers

Repository administrators or users with sufficient privileges can bypass software gates. We do not claim cryptographic enforcement against the human owner. Instead the scientific record distinguishes governed promotion from operator override. An override is explicitly logged, cannot self-create evolution evidence and triggers post-deployment containment/attestation work. This boundary is important for honest evaluation of real engineering practice, where emergencies and direct operator instructions exist.

10.11 Rollback as a first-class operation

Rollback is not failure erasure. A promoted variant retains the evidence that supported it, while new incidents can produce a new assessment that narrows or reverses the operational decision. The previous incumbent or another assured branch remains addressable in the variant DAG. Rollback tests bind state/schema compatibility and required active paths. If state migration is irreversible, that fact is itself a Class-B/C deployment risk that must be disclosed before promotion.

10.12 Security success criteria

For Paper 5, the strongest security claims are negative invariants rather than claims of perfect safety. The target measurements include zero invalid promotions in the registered tests, zero false independent-review credit, zero fresh-transfer state leakage, exact-subject evaluator binding, complete preservation of source episodes through lesson supersession, and successful detection of planted authority/identity attacks. A failure on one of these properties blocks a strong framework-evolution claim even if task performance improves.

11 Relation to agent memory, self-improvement and autonomous research

Paper 5 sits at the intersection of several rapidly developing areas. We therefore avoid a broad priority claim such as “the first self-improving research agent.” The contribution is a particular combination of research-process representation, longitudinal metrology, causal attribution and governed architecture evolution.

11.1 Learning from episodic experience

Reflexion showed that language agents can improve without parameter updates by storing verbal feedback in episodic memory and reusing it on later trials [2]. ExpeL similarly extracts

natural-language insights from collections of agent experiences and recalls them at inference time [3]. These systems establish the basic usefulness of external experiential memory.

RAKL v3 differs in the authority and lineage semantics of its memory. A raw `TaskEpisode` is an immutable evidence root. A `Lesson` is a versioned abstraction with support, contradiction, scope, falsifier and explicit authority. Failure observation, failure diagnosis and reusable obstruction are separate states rather than one reflection string. This separation is motivated by the possibility that a useful episode can support a faulty consolidation. Recent work reports exactly such a failure mode: repeated LLM consolidation can degrade memory utility even when retaining the underlying episodes remains competitive, motivating explicit gates around consolidation rather than mandatory rewriting after every interaction [9].

11.2 Workflows, procedural memory and reusable skills

Voyager demonstrated an ever-growing executable skill library with environment feedback and self-verification in Minecraft [4]. Agent Workflow Memory induces reusable workflows from past trajectories and reports transfer across tasks, websites and domains [5]. More recent systems such as MemSkill make memory-extraction and consolidation routines themselves evolvable, while other skill-centric systems manage creation, reuse and refinement across a skill lifecycle [10, 11].

Our question is complementary. We treat reusable workflows and skills as one view of a broader research state that also contains negative history, unresolved obstructions, compatibility relations, problem-conditioned fibres, gluing failures and meta-method variants. The paper further asks how to measure whether such accumulated structure changes research behavior relative to the same base model, and how a lesson about research practice can safely become a change to the framework that stores and uses future lessons.

11.3 Recursive research loops

AREX alternates deep research with constraint-wise auditing and targeted follow-up research, using a learned context-update tool to maintain a compact improvement state over long horizons [12]. This discovery–verification loop is close in spirit to obstruction-guided research. RAKL adds a distinct governance question: the verified state used to improve research is not automatically allowed to certify a modification of the research method itself. Method evolution goes through a separate challenger/evaluator/assurance/promotion path.

11.4 Automated design and recursively self-improving agents

Automated Design of Agentic Systems frames agent design itself as a searchable programming problem and demonstrates meta-agent discovery of transferable agent designs [13]. Gödel Agent makes the agent logic self-referential and editable [14]. Darwin Gödel Machine maintains an open-ended archive of self-modified coding agents and empirically selects improvements on coding benchmarks [16]; Hyperagents extend editable self-improvement to the mechanism that generates future agent changes [17]. These systems make clear that self-modifying code and open-ended variant archives are technically viable research objects.

The RAKL variant archive borrows the general lesson that one should preserve multiple branches rather than repeatedly overwrite a single incumbent. Our emphasis, however, is not unconstrained open-ended search. The evidence target is scientific research methodology, where outcome verification can be incomplete, novelty and truth are distinct, and evaluator contamination or authority overclaim is itself a failure. Accordingly, RAKL distinguishes development gain, fresh assurance, governance approval and post-deployment attestation, and it preserves operator overrides as non-validating operational events.

A recent Red Queen Gödel Machine explicitly studies co-evolving agents and evaluators under non-stationary utilities [18]. This creates an important comparison. We do not assume that evaluators must remain fixed forever. Instead, we freeze an evaluator within one evidence epoch and require evaluator evolution to occur as a separately governed transition between epochs. This prevents a candidate from changing the criterion that judges the same candidate while still permitting the utility surface to evolve prospectively.

11.5 Evaluator-guided scientific and mathematical discovery

AlphaEvolve combines LLM-generated code with iterative automated evaluation and has been applied to algorithmic and mathematical discovery [19, 20]. This work demonstrates that strong evaluators can turn model generation into productive large-scale search. Paper 4 of the RAKL programme addressed the adjacent problem of mathematical authority: proof, formalization, novelty, research value and verifier trust must not be collapsed merely because evaluator-guided search is powerful. Paper 5 moves one level upward and studies the research method as the object being learned.

11.6 Long-horizon behavioural studies of research agents

A particularly close 2026 study follows roughly one hundred sequential architecture experiments performed by a single LLM researcher and finds phase-structured productivity, a long saturation wall, recovery after expanding the action surface, workflow-induced greedy search and rediscovery of established results [1]. That result materially narrows the novelty claim available to this paper: observing a long-running agent and concluding that workflow design affects research behavior is not new by itself.

Our longitudinal case study is intended to add three elements. First, every consequential trajectory is represented in a typed evidence system that distinguishes raw episode, diagnosis, lesson, obstruction, reusable operator and framework variant. Second, the case study is coupled to explicit lattice/process metrology and matched model-only/reset/sham-memory/learning attribution rather than only retrospective behavioural interpretation. Third, recurring process pathologies can trigger content-identified framework challengers whose claimed improvement must pass protected evaluation and governance before becoming a new method version.

11.7 The specific claim of Paper 5

The contribution is therefore not any one ingredient in isolation. The paper studies the closed but non-self-certifying loop

research episode → bounded diagnosis → lesson/method hypothesis → framework challenger → matched evaluation → fresh protected assurance → governed promotion → new research episodes

while preserving the previous evidence and method variants. Whether this loop produces sustained improvements across several RAKL versions is an empirical question. The architecture and early case-study incidents are available now; the stronger longitudinal performance claim remains prospective.

12 Threats to validity, alternative explanations and decisive falsifiers

A paper about self-improving research systems is unusually vulnerable to circular evidence. We therefore state the strongest alternative explanations and the outcomes that would force claim narrowing.

12.1 The live case study is observational and selected

The six Millennium programmes were intentionally chosen because they are difficult, open and structurally diverse. They are not a random sample of scientific research. Their value is as a stress-test observatory, not as a population from which we can directly estimate the average behavior of all research agents. Furthermore, individual atomic tasks are selected adaptively by the agents and framework. Frequencies observed in the live archive therefore mix properties of the problem distribution with properties of the selection policy.

Paper 5 accordingly separates retrospective pathology counts from prospective benchmark results. Statements such as "surrogate/root-coordinate confusion recurs across several domains" are case-study observations. Population-level rates require a frozen sampling rule or benchmark set.

12.2 Open problems do not provide frequent final success labels

For an unsolved root problem there may be no positive terminal outcome during the study. This creates a risk that the framework rewards internally generated notions of progress. Residual contraction, lesson authority and local theorem validation are therefore reported as intermediate coordinates, never as substitutes for root success. A local lemma can be mathematically correct while contributing nothing to the root. The gluing/interface metrics explicitly measure this failure mode.

The strongest capability claims will therefore rely on auxiliary task distributions with externally checkable or held-out outcomes in addition to the live open-problem observatory.

12.3 The underlying model can be the real cause

An agent may solve a task because the language model already knew how. This is the motivation for the four-arm design in Section 9. If `MODEL_ONLY` performs as well as `RAKL_LEARNING` at matched validity and resources, the data do not support a RAKL capability benefit. If learning and sham-memory arms are similar, the content-specific memory hypothesis is not supported. If `RAKL_RESET` accounts for the gain, the effect belongs to static workflow scaffolding rather than continual experience.

12.4 Extra compute and context can masquerade as intelligence

RAKL performs retrieval, preprocessing, validation and memory compilation. An unbudgeted comparison against a shorter direct prompt would be unfair. All arms therefore share a preregistered resource ceiling, and actual usage is reported. The sham-memory arm additionally controls, imperfectly, for the possibility that merely adding more tokens or structured context improves performance.

No sham is perfect. Irrelevant memory may distract more than an empty context, while carefully chosen near-miss memory may create a harder condition. We will therefore freeze sham construction rules before task outcomes and report more than one sham stratum when sufficient budget is available.

12.5 Prompt and workflow differences are part of the intervention

A full RAKL-versus-model-only comparison necessarily changes instructions. It estimates the effect of the complete architecture, not a single isolated variable. The reset-versus-learning comparison is cleaner for experience because both arms use the same RAKL workflow. Component ablations then localize specific mechanisms. We will not interpret the total RAKL lift as a pure memory effect.

12.6 Model-service drift and non-determinism

Hosted model names can conceal backend changes, and repeated samples can differ even under nominally fixed settings. Exact model revision is recorded when available. Strong effects should be replicated across seeds/sampling configurations and, where the claim is intended to generalize, across more than one model family or revision. Failure to reproduce under a changed hosted model does not by itself distinguish framework failure from model drift, but it weakens claims of model-independent benefit.

12.7 Temporal dependence and pseudo-replication

Development episodes are not independent because later states contain earlier experience. Treating every cycle as an IID sample would underestimate uncertainty. The task is the unit for paired benchmark comparisons; fresh-transfer tasks all start from the same frozen learned state. Longitudinal curves remain descriptive unless the dependence structure is explicitly modelled. Multiple episodes generated from the same source or failure lineage are not counted as independent transfer validations.

12.8 Concurrent agents can contaminate lanes

Several research agents write to a shared application repository. One lane can therefore encounter artifacts produced by another lane between its start and completion. Exact base/head identities and fibre snapshots are required to establish what was available at decision time. A fresh-transfer benchmark uses frozen immutable states rather than a moving shared `main`. Uncontrolled cross-lane writes invalidate a benchmark packet but remain legitimate observational case evidence.

12.9 Telemetry changes the behavior being observed

Requiring agents to record failures, retrieved memory and saturation may itself improve or constrain behavior. This is not a defect if the telemetry is part of RAKL, but it matters for interpretation. The pre-telemetry archive and reset/model-only arms provide partial controls. If Paper 5 claims that observation alone reveals natural model behavior, that claim would be too strong. We study the behavior of agents operating inside a specified research architecture.

12.10 Semantic novelty labels can be wrong

The seven-axis growth vector requires identity and semantic-retention judgments. Raw counts are objective relative to the stored data; retained novelty can depend on imperfect canonicalization. We therefore preserve the raw object stream, version novelty criteria, permit later supersession and avoid using retained novelty as a sole promotion metric. Periodic external/manual audits of sampled novelty assignments are desirable.

12.11 Residual contraction can be gamed

A framework can make the blocker set smaller by hiding or coarsening obligations. This would create artificial progress. Residual transformations therefore preserve evidence and are cross-checked against root contracts and gluing coverage. A step that exposes new independent blockers is not penalized merely because the count increases. Hostile tests should include obstruction renaming and deliberate omission of an interface.

12.12 Retrieval recall requires a bound universe

A claim that no relevant memory was found is not measurable against an unbounded repository narrative. The XM003 case exposed exactly this problem. Retrieval recall is therefore reported

only relative to a content-identified index/universe and later relevance adjudication. A future framework-level coverage receipt is an explicit engineering requirement rather than an assumed property.

12.13 Same-session expert cells are not independent reviewers

The live research lanes use role-separated expert passes for adversarial coverage. These roles share model context and cannot be counted as independent peer review. The Paper 5 drafting/review loop in the present session has the same limitation. Nature-style concern taxonomies can improve coverage, but same-session internal review is labelled as such. Strong external-review claims require genuinely separate people/processes and evidence lineage where relevant.

12.14 Researcher and author intervention

The human operator chooses high-level goals, can change scheduled-agent prompts, may override merges and is co-designing the framework while observing it. The system is therefore not an experiment in fully autonomous AI development. Operator interventions are timestamped as part of the method history. Paper 5's claim concerns evidence-governed human/AI recursive research infrastructure, not human-free self-evolution.

12.15 The paper is being written while the system evolves

A live repository creates a moving-target risk. Every reported result therefore needs an explicit evidence cutoff and exact subject. Later findings can be added as new versions of the manuscript, but they cannot be silently backdated into the original evaluation. The present draft intentionally contains placeholders for prospective metrics rather than estimating them from incomplete future data.

12.16 Publication and confirmation bias

A self-improvement project naturally prefers stories of successful evolution. We counter this by preserving rejected variants, baseline-only wins, invalid transfers, failed consolidations, blocked promotions and operational overrides. Paper 5 should publish a complete registered evaluation packet, not only the tasks on which RAKL helps. If the result is null or negative, the framework and paper claims must narrow accordingly.

12.17 Security and evaluator independence are assumptions, not proofs

Content addressing and protected paths do not protect against a compromised repository administrator, CI service, dependency or assurance operator. The no-self-promotion proposition is conditional on these boundaries. Threats outside the trusted computing base are reported rather than treated as solved by hashes.

12.18 Decisive result branches

The prospective programme is designed so that important hypotheses can fail cleanly.

H1: accumulated RAKL experience improves fresh research. Supported only if `RAKL_LEARNING` shows a material preregistered gain over `RAKL_RESET` on fresh transfer without a blocking validity/resource regression. Refuted for the tested distribution if the effect is materially negative. Inconclusive if confidence/coverage is insufficient.

H2: learned memory content, not extra context, causes part of the gain. Supported only if learning materially outperforms preregistered sham-memory controls. If sham and learned memory are equivalent, the content-specific claim is unsupported even if both outperform reset.

H3: RAKL helps beyond the base LLM. Supported in scope only if total matched lift over `MODEL_ONLY` is positive on the registered primary QoIs or if a separately preregistered epistemic-safety benefit is material. A substantial excess of `BASELINE_ONLY_SUCCESS` tasks is evidence against the claim.

H4: experience reduces repeated structural failure. Supported if fresh-task repeated-failure rate falls without increasing false-transfer or invalid-authority rates. If the framework simply avoids old failures by making different but equally serious failures, the hypothesis is not supported.

H5: framework evolution is more than benchmark overfit. Supported for a particular parent–child edge only after a preregistered development gain transfers to a fresh, protected, lineage-independent assurance packet under matched resources. Development-only improvement is explicitly classified as local improvement rather than evolution evidence.

H6: RAKL's operator/representation atlas approaches a high zero-invention regime. This RAKL-triviality hypothesis remains open. A growing share of verified tasks solved with only preexisting resources would support it; a persistent need for new representations/operators/ontology across fresh tasks would refute or bound it.

These result branches are part of the method. They prevent future manuscript revisions from defining "success" after observing whichever metrics happen to improve.

13 An evidence-triggered roadmap from RAKL 3.0 to later challengers

A roadmap can itself become a source of confirmation bias if version numbers are assigned to ideas before evaluation. We therefore distinguish *candidate sequence* from *released method sequence*. A label such as 3.1 is earned only after the corresponding challenger passes its governed process. If the first candidate fails, it remains a rejected branch and a different challenger may eventually become 3.1.

13.1 3.0.x: measurement and authority hardening without claiming a new research method

Several immediate changes are instrumentation or implementation hardening rather than intended research-policy innovations. Suitable patch candidates include:

- read-only state/process metrology and the four-arm attribution data structures introduced on the Paper 5 branch;
- content-bound replacement for the v3 archive's bare governance Boolean while preserving the existing constitutional requirement that promotion needs governance approval;
- content resolution of verification/review/proof IDs before authority-bearing use;
- certificate-backed local-section verification on gluing authority paths;
- promotion-grade binding of benchmark state/output identities to exact task/evaluator/artifact subjects;

- public API and CI stabilization.

These repairs should improve observability and enforcement of already stated principles. Unless they intentionally change search behavior, they do not by themselves support a new method-generation claim.

13.2 First Class-B candidate: coverage-bound research memory

The strongest currently observed cross-problem process defect is the XM002 retrieval miss: a synthesis stated that a Hodge reuse remained absent even though the exact base already contained the corresponding ResearchMemoryReview and reuse artifacts. Framework issue `RAKL#119` therefore proposes a content-bound cross-problem coverage receipt.

The candidate mechanism is straightforward. A memory query binds the exact problem/lane universe, index/manifests and structural query coordinates before making completeness-like statements. The primary prediction is lower missed-relevant-memory rate without requiring full-repository loading. Secondary predictions include lower duplicate research and better cross-domain tool reuse. Main risks are context/compute overhead and false confidence in an incomplete index.

The cheapest discriminating benchmark uses frozen repository snapshots with planted relevant and near-miss memories across several lanes, comparing the current query path with the coverage-bound challenger at matched retrieval budget. A successful development result still requires fresh surface-shifted tasks before the candidate can become a new method version.

13.3 Root-coordinate preservation audit

Several live case-study episodes exhibit a shared abstraction: an appealing local or surrogate quantity does not preserve the root-critical coordinate. Examples include raw closure mass versus cover complexity, positive norm versus Li-coefficient faithfulness, augmentation order versus complex analytic order, finite-cutoff spectral information versus continuum physical gap, and detector visibility versus actual geometric obstruction vanishing.

If the pattern survives enough independent episodes, a Class-B challenger can require an explicit `RootCoordinatePreservationAudit` before a surrogate becomes a major route. The intervention asks for the exact map from surrogate to root coordinate, non-preservation examples, cheapest hostile control and remaining gluing obligations. Its primary metric is reduction in root-coordinate false-progress/candidate-generation rate. Its main risk is excessive conservatism that blocks useful exploratory representations before a bridge can be discovered.

13.4 Gluing-aware search policy

Another recurring pattern is local mathematical success with an unresolved global interface. Yang–Mills provides an abstract spectral lemma whose target-specific OS/density/continuum binding remains open; Navier–Stokes repeatedly exposes far-field, compactness or limit-passage interfaces after valid local estimates. V3 represents gluing obstructions, but a later challenger could make interface risk an explicit routing coordinate.

The predicted benefit is earlier detection of "correct but non-closing" local work and higher residual contraction per cycle. The hostile control is a problem where local decomposition is sufficient and extra gluing machinery only adds overhead. The candidate should fail if it systematically diverts effort from locally solvable tasks.

13.5 Experience-conditioned routing and strategy motifs

V3 already contains experience-conditioned operator scoring and motif induction. The correct next action is evaluation, not automatic elaboration. Once enough v3 telemetry accumulates,

compare routing with and without experiential priors on repeated-family, transfer and hostile near-miss tasks. A successful result should reduce repeated failures and/or cost while maintaining false-transfer control. If gains disappear under sham memory or fresh transfer, the mechanism should remain local or be revised rather than promoted as a general method improvement.

13.6 Consolidation policy as an evolvable object

The v3 fast/slow split intentionally prevents every episode from being immediately rewritten into a lesson. Later data may support different consolidation schedules by failure type, task family or evidence density. This is a natural candidate for method learning because external research has demonstrated that forced continuous consolidation can degrade memory. Any challenger must preserve raw episodes and be compared against episodic-only and current gated-consolidation controls.

13.7 Portfolio and saturation policy

The long-horizon autonomous-research literature and our own saturation vector both suggest a failure mode in which an agent repeatedly hill-climbs within one method family after returns flatten. A future challenger could use measured axis-specific saturation and stable residuals to reallocate budget among exploitation, diversification, moonshots and meta-method search. The benchmark must include tasks where persistence within one family is actually correct so that diversification is not rewarded merely for being novel.

13.8 Meta-method invention

Only after relevant operator/path/meta-method axes are demonstrably saturated and stable residuals remain should the system search for a genuinely new research operator or ontology. The v3 invention gate operationalizes this principle. Paper 5 treats operator invention as a measured exception rather than the default explanation of progress. This creates an empirical test of the RAKL-triviality hypothesis: how often do verified solutions require new primitive research structure rather than recomposition or transfer?

13.9 What later versions should publish

Each released method version should have a compact "model card" style evidence packet:

- exact parent and candidate Git/method identities;
- source pathologies or positive motifs motivating the change;
- preregistered mechanism and predicted metrics;
- matched development results and uncertainty;
- fresh protected assurance results;
- regressions and baseline-only wins;
- resource profile;
- state/schema migration and rollback status;
- evaluator/assurance identities and exposure history;
- final governance/promotion and active-main attestation.

Paper 5 can then become a longitudinal scientific record of the sequence

$$\mathrm{RAKL}_{3.0} \to \mathrm{RAKL}_{3.1} \to \mathrm{RAKL}_{3.2} \to \cdots$$

without assuming in advance that every arrow points to a better system.

14 Discussion

The central proposal of Paper 5 is that the trajectory of a research agent is itself a scientific object. A long-running agent does not merely output candidate theorems, code or literature summaries. It reveals which representations attract search, which memories are retrieved or missed, which failures recur, which local successes fail to compose, when verification changes the route, and which engineering constraints dominate the observed behaviour. If these traces are recorded as typed evidence rather than rewritten as narrative reflection, they can support explicit hypotheses about the research method.

This view changes the purpose of failure. In a conventional benchmark, a failed attempt is often discarded after scoring. In an open research programme, the same failure may be the most reusable result of the cycle. The live Millennium case study has already produced examples in which a route was narrowed because a surrogate failed to preserve the root coordinate, a source could not support the assumed transformation, a local theorem left the real interface open, or the research process itself missed relevant prior memory. None solves the root problem, but each can prevent future agents from spending identical effort under equivalent assumptions.

At the same time, storing failure is not sufficient. The strongest caution from both our architecture and recent memory research is that an abstraction can be worse than the episodes from which it was produced. RAKL therefore makes lesson formation, failure diagnosis and tool promotion explicit transformations with contradiction, scope and validation obligations. The framework is designed to learn without requiring every learning event to become authoritative.

14.1 Why metrology matters for claims about augmented intelligence

The four-arm benchmark is perhaps the most important methodological addition for interpreting future results. "An LLM with RAKL solved the task" is not evidence that RAKL caused the solution. The model may already solve it; the framework may simply add tokens; a static checklist may explain the gain; or learned memory content may genuinely redirect search. Separating model-only, reset, sham-memory and learned conditions makes these explanations empirically distinguishable in a scoped task distribution.

This distinction is particularly important for scientific AI, where final success labels are sparse. A framework may contribute primarily by detecting false progress, reducing repeated failures or finding the missing interface earlier. These are legitimate effects, but they should be reported as epistemic-safety or efficiency effects rather than converted into a claim that the system became a stronger mathematician.

14.2 Research architecture as an evolving experimental treatment

We propose treating each method version as an intervention. RAKL 3.1 should not be a marketing label for a larger repository; it should identify a challenger motivated by specific episodes, with a frozen mechanism and measurable prediction. This creates a sequence of natural experiments in which the method that generated the previous research record is itself changed and reevaluated.

The approach resembles automated agent design and self-modifying-agent work in maintaining variants and using empirical evaluation, but the scientific setting imposes additional constraints. The evaluator may be incomplete, truth and novelty are separate, fresh tasks are expensive, and a system capable of editing its own methods must not be allowed to redefine evidence after

seeing outcomes. Protected assurance and governance therefore remain outside the candidate's authority.

14.3 What AI engineers may learn from the case study

For AI-system engineers, the project exposes a practical split between model capability and harness capability. Repository-native instructions, context compilation, retrieval coverage, state versioning, exact-subject CI, state migration and rollback are not peripheral details. They shape what the agent actually sees and what actions remain available. The live post-v3 CI repair is a useful example: a method-level architecture can be conceptually rich while repository integration still fails on mundane but load-bearing artifact and workflow details. Engineering incidents belong in the same longitudinal evidence system because they constrain reproducibility and deployment.

14.4 What formal-methods and assurance researchers may ask

The strongest assurance question is not whether every research decision can be proved optimal. It cannot. The more tractable question is whether authority escalation is structurally separated from generation. Paper 4 developed this idea for mathematical claims; Paper 5 applies the same philosophy to method evolution. A challenger may propose its own successor and may even discover an evaluator weakness, but a strong promotion claim depends on content-bound evidence and protected decision points outside that proposal path.

This architecture does not eliminate the trusted computing base. It makes it explicit. A compromised human administrator, CI service or governance credential can still override the system. The scientific claim is therefore scoped to observable process separation rather than absolute self-governance.

14.5 What knowledge-representation researchers may ask

The common substrate deliberately avoids treating all knowledge as one homogeneous graph. Epistemic claims, operators, episodes, obstructions and meta-method variants overlap in identity space while retaining specialized semantics and authority. Growth is measured on typed axes because a new relation and a new theorem are not interchangeable units. This may also make the longitudinal archive useful for studying whether reusable operator/strategy spaces saturate faster than the problem distribution.

The RAKL-triviality hypothesis is intentionally empirical. If many verified tasks eventually require only stored, compositional or explicitly transferred structure, then increasing the atlas may shift effort from invention toward retrieval and gluing. If fresh tasks continue to demand new representations, operators or ontology, the hypothesis will be bounded or rejected. Either outcome informs the architecture.

14.6 What scientists and mathematicians may ask

The Millennium programmes are not presented as evidence that autonomous agents are close to solving the problems. Their role is to supply long-horizon, high-stakes research behaviour in domains where superficial progress is easy to overstate. Exact root contracts, primary-source discipline, typed failures, same-context-versus-independent review separation and explicit local-to-global obligations are intended to make this behaviour auditable.

A mathematically correct local lemma remains valuable even when it does not close the root. Conversely, an attractive representation with no faithful bridge to the root is treated as a research risk. The case study therefore produces a record not only of attempted solutions but of the methods by which a research system learns where not to place confidence.

14.7 The empirical programme can fail productively

Several plausible outcomes would force revision of our current design. Experience may cause fixation rather than improvement; lesson consolidation may harm more often than it helps; cross-domain memory may increase false transfer; the process overhead may exceed any saved search; or static RAKL scaffolding may account for nearly all observed gains. A version may improve one domain while harming another. A coverage receipt may cost too much context. A root-coordinate audit may become excessively conservative. These are not loopholes in the paper; they are registered branches of the experiment.

The framework should therefore evolve only when the new evidence identifies a change whose benefit survives fresh testing. Recursive improvement without the possibility of a stable null result is not a scientific loop.

15 Conclusion

We introduced an evidence-governed formulation of continual research-method evolution. RAKL v3 represents immutable research episodes, derived lessons, failures, operators, motifs, problem fibres, gluing obstructions, saturation coordinates and framework variants as typed overlapping views of persistent external state. A live multi-agent mathematical programme provides an observational case study of the methods agents use, the ways they fail and the process defects they expose. The present evidence already motivates concrete framework hypotheses, but does not establish that a later version is superior.

To make that claim testable, Paper 5 contributes a governed upgrade protocol, repository-native coding-agent harness, typed process metrology, seven-axis lattice-growth measurement, four-arm causal-attribution benchmark, protected fresh assurance, operator-override semantics, variant archives and post-promotion attestation. Together these components define the experimental unit for future RAKL evolution:

$$\boxed{\text{observe} \to \text{diagnose} \to \text{propose} \to \text{freeze} \to \text{implement} \to \text{compare} \to \text{assure} \to \text{govern} \to \text{observe again}}.$$

The next scientific result is therefore not predetermined. If future matched data show that RAKL 3.1 or 3.2 reduces repeated research failures, improves fresh transfer, lowers false progress or reaches validated results more efficiently than the same underlying model and prior method under matched conditions, the evidence will support a scoped evolution claim. If they do not, the corresponding challenger should remain rejected or local. In either case, the longitudinal archive becomes an empirical record of how an AI-assisted research method changes, what it learns, and where its own theory of research fails.

Code, data, materials and AI-use disclosure

The public framework and manuscript sources are maintained at `https://github.com/SzeChunYiu/RAKL`; the mathematical application case-study artifacts are maintained separately at `https://github.com/SzeChunYiu/RAKL_math`. Research agents, coding agents and language models were used as proposal, implementation, literature-search and drafting tools. They are not represented as independent reviewers, human experts or authors. Public research traces are auditable decision records, not disclosures of hidden model chain-of-thought. The longitudinal mathematical programme studies unsolved problems and carries no claim that any Millennium Prize Problem has been solved unless a separate root certificate satisfies the applicable proof, dependency, novelty and independent-review gates. Prospective evaluation quantities that have not yet been measured are marked as preregistered targets or placeholders rather than reported as results.

References

[1] A. Safdar and M. Saadeldin, "Long-Horizon Autonomous Architecture Research with a Language-Model Agent: A Behavioural Case Study," *arXiv:2608.01995* (2026).

[2] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," *arXiv:2303.11366* (2023).

[3] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, "ExpeL: LLM Agents Are Experiential Learners," *arXiv:2308.10144* (2023).

[4] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An Open-Ended Embodied Agent with Large Language Models," *arXiv:2305.16291* (2023).

[5] Z. Z. Wang, J. Mao, D. Fried, and G. Neubig, "Agent Workflow Memory," *arXiv:2409.07429* (2024).

[6] C. Wang, Z. Yu, X. Xie, W. Yao, R. Fang, S. Qiao, K. Cao, G. Zheng, X. Qi, P. Zhang, and S. Deng, "AutoSkills: Automatically Constructing Skill Knowledge Bases for Agents," ICML 2026 submission/accepted paper, OpenReview id `rJS7Z3Oaw1` (2026).

[7] G. Jiang, Z. Su, X. Qu, and Y. R. Fung, "XSkill: Continual Learning from Experience and Skills in Multimodal Agents," *arXiv:2603.12056*; ICML 2026 (2026).

[8] Q. Hu, Q. Long, and W. Wang, "When Continual Learning Moves to Memory: A Study of Experience Reuse in LLM Agents," *arXiv:2604.27003* (2026).

[9] D. Zhang, Y. Lin, Z. Wu, Y. Sun, B. Li, D. Li, and H. Peng, "Useful Memories Become Faulty When Continuously Updated by LLMs," *arXiv:2605.12978* (2026).

[10] H. Zhang, Q. Long, J. Bao, T. Feng, W. Zhang, H. Yue, and W. Wang, "MemSkill: Learning and Evolving Memory Skills for Self-Evolving Agents," *arXiv:2602.02474* (2026).

[11] H. Lin, P. Li, J. Song, F. Jiang, and T. Zhang, "MUSE-Autoskill: Self-Evolving Agents via Skill Creation, Memory, Management, and Evaluation," *arXiv:2605.27366* (2026).

[12] S. Lu et al., "AREX: Towards a Recursively Self-Improving Agent for Deep Research," *arXiv:2607.21461* (2026).

[13] S. Hu, C. Lu, and J. Clune, "Automated Design of Agentic Systems," *arXiv:2408.08435* (2024).

[14] X. Yin, X. Wang, L. Pan, X. Wan, and W. Y. Wang, "Gödel Agent: A Self-Referential Agent Framework for Recursive Self-Improvement," *arXiv:2410.04444* (2024).

[15] X. Yin, X. Wang, L. Pan, X. Wan, and W. Y. Wang, "Gödel Agent: A Self-Referential Agent Framework for Recursive Self-Improvement," *arXiv:2410.04444* (2024).

[16] J. Zhang, S. Hu, C. Lu, R. Lange, and J. Clune, "Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents," *arXiv:2505.22954* (2025).

[17] J. Zhang, B. Zhao, W. Yang, J. Foerster, J. Clune, M. Jiang, S. Devlin, and T. Shavrina, "Hyperagents," *arXiv:2603.19461* (2026).

[18] A. Iacob et al., "The Red Queen Gödel Machine: Co-Evolving Agents and Their Evaluators," *arXiv:2606.26294* (2026).

[19] A. Novikov et al., "AlphaEvolve: A coding agent for scientific and algorithmic discovery," *arXiv:2506.13131* (2025).

[20] B. Georgiev, J. Gómez-Serrano, T. Tao, and A. Z. Wagner, "Mathematical exploration and discovery at scale," *arXiv:2511.02864* (2025).

[21] OpenAI, "Introducing Codex," OpenAI technical/product article (2025), `https://openai.com/index/introducing-codex/`.

[22] OpenAI, "Harness engineering: leveraging Codex in an agent-first world," OpenAI engineering article (2026), `https://openai.com/index/harness-engineering/`.

[23] S. C. Yiu, "Verified Discovery: An Assurance Architecture for LLM-Mediated Mathematical Research," RAKL Paper 4 source package (2026), `https://github.com/SzeChunYiu/RAKL/tree/main/publication/papers/paper-04-verified-discovery`.

[24] B. P. Cavalar and I. C. Oliveira, "Boolean Circuit Complexity and Two-Dimensional Cover Problems," *ACM Transactions on Computation Theory* **17**(2), Article 13 (2025), doi:10.1145/3718746; preprint *arXiv:2503.14117*.

[25] M. Suzuki, "Li coefficients as norms of functions in a model space," *Journal of Number Theory* **252**, 177–194 (2023), *arXiv:2301.05779*.

[26] K. Ota, "Kato's Euler system and the Mazur–Tate refined conjecture of BSD type," *arXiv:1509.00682* (2015).

[27] G. Seregin, "On potential Type II blowups for the Navier–Stokes equations," *arXiv:2606.29468* (2026).

[28] T. Bałaban, "Averaging operations for lattice gauge theories," *Communications in Mathematical Physics* **98**(1), 17–51 (1985), doi:10.1007/BF01211042.

[29] RAKL_math, "research(cross): record Hodge third-domain transfer and retrieval miss," Pull Request 50 and associated XM003 artifacts (2026), `https://github.com/SzeChunYiu/RAKL_math/pull/50`.

[30] RAKL, "Framework: bind cross-problem memory coverage before completeness claims," Issue 119 (2026), `https://github.com/SzeChunYiu/RAKL/issues/119`.

[31] RAKL_math, "research(xm): reject raw PNP closure volume as hardness coordinate," Pull Request 64 and associated XM004 artifacts (2026), `https://github.com/SzeChunYiu/RAKL_math/pull/64`.

[32] RAKL_math, "research(rh): freeze RH-ANA-002 Li-faithfulness packet," Pull Request 39 / main commit `1a6add16...` (2026), `https://github.com/SzeChunYiu/RAKL_math/pull/39`.

[33] RAKL_math, "research(rh): quarantine RH-SPEC-003 endpoint defect and repair assurance," Pull Request 53 (2026), `https://github.com/SzeChunYiu/RAKL_math/pull/53`.

[34] RAKL_math, "research(bsd): isolate S001c1a complex upper-bound obstruction," Pull Request 44 (2026), `https://github.com/SzeChunYiu/RAKL_math/pull/44`.

[35] RAKL_math, "research: prove YM-S1a1 dense common-rate spectral exclusion," Pull Request 62 (2026), `https://github.com/SzeChunYiu/RAKL_math/pull/62`.

[36] RAKL_math, "[NS-B2a] Bind Seregin 2606.29468 Type-II Euler-limit rigidity interface," Issue 65 (2026), `https://github.com/SzeChunYiu/RAKL_math/issues/65`.

[37] RAKL_math, "research(ym): bind YM-E1a1a0 primary-source retrieval boundary," Pull Request 59 (2026), https://github.com/SzeChunYiu/RAKL_math/pull/59.

[38] RAKL, "Merge RAKL v3 recursive experience substrate," commit a151d5612709ea0f95c3ea232630f246f722739a (2026), https://github.com/SzeChunYiu/RAKL/commit/a151d5612709ea0f95c3ea232630f246f722739a.

[39] RAKL, "fix(ci): purge symlinks and restore frozen Paper 1 builder after 117/118 merge," Pull Request 120 (2026), https://github.com/SzeChunYiu/RAKL/pull/120.

Case	Observable event	Process diagnosis	Framework hypothesis
Cross-problem XM003	a synthesis missed an already-present Hodge reuse of a registered cross-domain audit tool	retrieval coverage failure, not chronology failure	completeness/no-match claims need a bound coverage receipt
P-vs-NP XM004	raw source-native closure cardinality becomes maximal in a source-defined zero-cover-complexity regime	surrogate representation mass does not preserve root-critical cost	require a root-coordinate preservation audit before scoring a representation
RH analytic/spectral	positive or trace-like surrogate objects hide RH-equivalent faithfulness/endpoint obligations	representation and faithfulness defect	make bridge defect explicit and falsify the interface before theorem invention
BSD S001c1a	augmentation order contains local corrections and extra zeros unrelated to the desired complex rank coordinate	scalar representation is not root-faithful	rotate to corrected or richer graded objects rather than absorb excess vanishing by notation
Yang–Mills S1a1	an abstract dense-source spectral lemma is valid, but target OS source binding, uniformity, RG transport and continuum identification remain open	local theorem success with unresolved global interfaces	treat local-to-global gluing obligations as first-class residuals
Navier–Stokes B2a	a Type-II route yields an ancient Euler limit, making ordinary parabolic Navier–Stokes backward uniqueness inapplicable	unsafe method transfer across changed equation/interface	require source-to-target inheritance witnesses before reusing rigidity tools
Yang–Mills E1a1a0	primary metadata identified the correct Bałaban averaging source, but accessible text did not expose the exact nonlinear map/contract	source-completeness failure	missing source detail should block candidate generation rather than be reconstructed from memory
RAKL v3 merge incident	v3 was merged at operator request before CI cleanup; a later PR repaired CI and a benchmark float assertion	deployment occurred before assurance closure	deployment and method authority need separate states plus explicit operator-override records

Table 2: Initial qualitative cases. These are hypothesis-generating observations, not estimates of prevalence or causal treatment effects.